



UNIVERSITÀ DEGLI STUDI  
DI NAPOLI FEDERICO II

# Progetto di Ingegneria del Software

---



## Candidati

Christian Casucci – N86004569  
Antonio Fusaro – N86004619  
Luigi Differente – N86004503

*Anno Accademico: 2025-2026*



# Indice

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione e contenuto del documento</b>          | <b>4</b>  |
| 1.1      | Contenuto Capitolo 2                                   | 4         |
| 1.2      | Contenuto Capitolo 3                                   | 4         |
| 1.3      | Contenuto Capitolo 4                                   | 5         |
| <b>2</b> | <b>Documento dei requisiti</b>                         | <b>6</b>  |
| 2.1      | Glossario                                              | 6         |
| 2.2      | Personas                                               | 7         |
| 2.2.1    | Personas 1 : Marco Bianchi                             | 8         |
| 2.2.2    | Personas 2 : Matteo Gentile                            | 9         |
| 2.2.3    | Personas 3 : Giulia Marino                             | 10        |
| 2.2.4    | Personas 4 : Sara Conti                                | 11        |
| 2.2.5    | Personas 5 : Elena Ferri                               | 12        |
| 2.2.6    | Analisi delle personas                                 | 13        |
| 2.3      | Casi d'uso - Use Case Diagram                          | 15        |
| 2.3.1    | Analisi del Diagramma dei Casi d'Uso                   | 15        |
| 2.4      | Mock-up a bassa fedeltà                                | 17        |
| 2.5      | Descrizione requisiti funzionali - Tabelle di Cockburn | 18        |
| 2.5.1    | Use Case 1 - Inserimento Nuova Issue                   | 18        |
| 2.6      | Requisiti non funzionali e di sistema                  | 20        |
| 2.6.1    | Requisiti non funzionali                               | 20        |
| 2.6.2    | Requisiti di Dominio                                   | 21        |
| <b>3</b> | <b>System Design</b>                                   | <b>22</b> |
| 3.1      | Architettura                                           | 23        |
| 3.1.1    | Descrizione dell'architettura                          | 23        |
| 3.2      | Commento e descrizione delle tecnologie                | 26        |
| 3.3      | Persistenza dei Dati                                   | 26        |
| 3.3.1    | Class Diagram non ristrutturato                        | 27        |
| 3.3.2    | Class Diagram ristrutturato                            | 28        |
| 3.3.3    | Modello Logico                                         | 29        |
| 3.3.4    | Modello fisico                                         | 30        |
| 3.4      | Business Layer                                         | 38        |
| 3.4.1    | Stack Tecnologico                                      | 38        |
| 3.4.2    | Architettura e Definizione di Servizio                 | 38        |
| 3.4.3    | Definizione architetturali dei moduli                  | 39        |
| 3.5      | Login-Service                                          | 39        |
| 3.5.1    | Workflow di Autenticazione e Sicurezza                 | 39        |
| 3.5.2    | Architettura: Riutilizzabilità e Portabilità           | 39        |
| 3.5.3    | Struttura dei Componenti                               | 40        |
| 3.5.4    | Diagramma UML LoginService                             | 40        |
| 3.6      | InserimentoUtente-Service                              | 41        |
| 3.6.1    | Validazione e Integrità dei Dati                       | 41        |



|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 3.6.2    | Architettura e Design Pattern                          | 41        |
| 3.6.3    | Diagramma UML InserimentoUtenteService                 | 42        |
| 3.7      | InserimentoIssue-Service                               | 43        |
| 3.7.1    | Modellazione del Dominio e Type Safety                 | 43        |
| 3.7.2    | Tracciabilità e Audit: L'Entità Cronologia             | 43        |
| 3.7.3    | Pattern Creazionali: Builder Pattern                   | 43        |
| 3.7.4    | Gestione della Transazionalità e Persistenza           | 44        |
| 3.7.5    | Diagramma UML InserimentoIssueService                  | 44        |
| 3.8      | RecuperoIssue-Service                                  | 45        |
| 3.8.1    | Ottimizzazione delle Query e Paginazione               | 45        |
| 3.8.2    | Aggregazione e Composite View                          | 45        |
| 3.8.3    | Gestione delle Relazioni: Eager vs Lazy Loading        | 45        |
| 3.8.4    | Diagramma UML RecuperoIssueService                     | 46        |
| 3.9      | ModificaIssue-Service                                  | 47        |
| 3.9.1    | Architettura Modulare e Libreria Condivisa (JwtCommon) | 47        |
| 3.9.2    | Logica di Business: Change Detection e Auditing        | 47        |
| 3.9.3    | Diagramma UML ModificaIssueService                     | 48        |
| 3.10     | CommentaIssue-Service                                  | 49        |
| 3.10.1   | Gestione dell'Identità: Un Approccio Ibrido            | 49        |
| 3.10.2   | Regole di Business e Validazione di Stato              | 49        |
| 3.10.3   | Ottimizzazione delle Relazioni JPA (Proxy Pattern)     | 49        |
| 3.10.4   | Diagramma UML CommentaIssueService                     | 50        |
| 3.11     | Libreria Condivisa: JWT-Common                         | 51        |
| 3.11.1   | Centralizzazione della Logica Crittografica            | 51        |
| 3.11.2   | Gestione delle Eccezioni                               | 51        |
| 3.11.3   | Integrazione                                           | 51        |
| 3.11.4   | Diagramma UML JwtCommon                                | 52        |
| 3.12     | Livello API Gateway                                    | 53        |
| 3.12.1   | Implementazione con Spring Cloud Gateway               | 53        |
| 3.12.2   | Deployment e Configurazione Dinamica su Azure          | 53        |
| 3.13     | Il Livello di Presentazione: Frontend Desktop          | 54        |
| 3.13.1   | Architettura Model-View-Controller (MVC)               | 54        |
| 3.13.2   | Comunicazione e Gestione dello Stato                   | 54        |
| 3.13.3   | Mockup ad Alta Fedeltà e Flusso Utente                 | 55        |
| 3.14     | Design dell'Esperienza Utente (UX) e Usabilità         | 56        |
| 3.14.1   | Validazione Reattiva e Feedback Visivo                 | 56        |
| 3.14.2   | Efficienza e Accessibilità da Tastiera                 | 56        |
| 3.14.3   | Interfacce Dinamiche (Progressive Disclosure)          | 56        |
| 3.14.4   | Gestione dello Stato e Integrità                       | 56        |
| <b>4</b> | <b>Processo di Sviluppo e Qualità del Software</b>     | <b>57</b> |
| 4.1      | Strumenti di versioning                                | 57        |
| 4.1.1    | Code Frequency                                         | 57        |
| 4.1.2    | Statistiche commit                                     | 58        |
| 4.1.3    | Statistiche commit per collaboratore                   | 59        |
| 4.2      | Strumenti di Qualità e Scelte Architettureali          | 60        |
| 4.2.1    | Analisi Statica con SonarQube e Docker                 | 60        |
| 4.2.2    | Gestione dei Falsi Positivi: Il caso dei DTO           | 60        |
| 4.2.3    | Risultati analisi                                      | 60        |
| 4.2.4    | Copertura dei Test (Code Coverage)                     | 61        |
| 4.2.5    | Warning "Missing Blame Information"                    | 61        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>5</b> | <b>Fase di Testing</b>                                   | <b>62</b> |
| 5.1      | Analisi delle Classi di Equivalenza . . . . .            | 62        |
| 5.2      | Test Plan: Modifica Issue . . . . .                      | 62        |
| 5.3      | Test di Unità . . . . .                                  | 63        |
| 5.3.1    | Test 1: Modifica Issue . . . . .                         | 63        |
| 5.3.2    | Test 2: Inserimento Commento . . . . .                   | 67        |
| 5.4      | Testing dell'Interfaccia Grafica (GUI Testing) . . . . . | 70        |
| 5.5      | Test di Usabilità (Validazione Black Box) . . . . .      | 71        |
| 5.5.1    | Protocollo e Scenario del Test . . . . .                 | 71        |
| 5.5.2    | Partecipanti e Obiettivi . . . . .                       | 72        |
| 5.5.3    | Risultati e Osservazioni . . . . .                       | 72        |
| 5.5.4    | Nuova analisi SonarQube . . . . .                        | 73        |





# Capitolo 1

## Introduzione e contenuto del documento

BugBoard26 è una piattaforma per la gestione collaborativa di issue in progetti software. Il sistema consente a team di sviluppo di segnalare problemi relativi a un progetto, monitorarne lo stato, assegnarli a membri del team e tenere traccia delle attività di risoluzione. Il sistema deve consistere in un'applicazione (mobile, desktop o web-based) performante e affidabile, attraverso cui gli utenti possono fruire delle funzionalità in modo intuitivo e rapido.

### 1.1 Contenuto Capitolo 2

Il seguente capitolo si configura come un documento di specifica dei requisiti software e contiene :

- Glossario
- Personas
- Use Case Diagram
- Mock-up a bassa fedeltà per un caso d'uso selezionato
- Descrizione testuale strutturata mediante formalismo tabellare (Tabella di Cockburn) dello stesso caso d'uso analizzato nel punto precedente
- Analisi dei requisiti non funzionali e di sistema

### 1.2 Contenuto Capitolo 3

Il seguente capitolo si configura come un documento di design del sistema e processo di sviluppo e contiene :

- Descrizione dell'architettura proposta, con esplicita definizione dei criteri di design adottati e delle motivazioni dietro tale scelta.
- Descrizione e motivazione delle scelte tecnologiche adottate.
- Descrizione dello schema per la persistenza dati.
- Diagramma delle classi di design.
- Descrizione e motivazione delle scelte di software design adottate.



## 1.3 Contenuto Capitolo 4

Il seguente capitolo ha il compito di evidenziare l'uso di strumenti di versioning e di report di qualità del codice e contiene :

- Evidenza dell'uso di strumenti di versioning.
- Report di qualità del codice generato da SonarQube.



## Capitolo 2

# Documento dei requisiti

### 2.1 Glossario

- **BugBoard26** : piattaforma per la gestione collaborativa di issue in progetti software. Il sistema consente a team di sviluppo di segnalare problemi relativi a un progetto, monitorarne lo stato, assegnarli a membri del team e tenere traccia delle attività di risoluzione.
- **Issue** : Segnalazione registrata all'interno del sistema, utilizzata per descrivere un problema, una richiesta o un'attività da svolgere in un progetto software. Ogni issue ha almeno un titolo e una descrizione, può avere priorità, tipologia , allegati e uno stato che ne indica l'avanzamento.
- **Team di sviluppo** : insieme di utenti che collaborano tra di loro per la realizzazione di un progetto software.
- **Utente** : persona fisica che accede mediante credenziali a BugBoard26
- **Utente amministratore** : utente il quale gode di maggiori opportunità quali ad esempio la registrazione di nuovi utenti.
- **Autenticazione** : Processo di verifica dell'identità di un utente tramite email e password
- **Bug** : Tipo di issue che segnala un malfunzionamento del software.
- **Question** : Tipo di issue usata per richiedere chiarimenti o informazioni aggiuntive.
- **Documentation** : Tipo di issue che segnala problematiche relative alla documentazione di un progetto.
- **Feature** : Tipo di issue che suggerisce eventuali sviluppi futuri per l'applicazione come ad esempio un'implementazione di una nuova funzionalità
- **Cronologia modifiche** : Registro che mostra chi ha modificato una issue e quando, inclusa la creazione.
- **Stato di una issue** : Indica la fase corrente del ciclo di gestione della issue, descrivendone l'avanzamento nel tempo.



## 2.2 Personas

L'analisi dei soggetti coinvolti nello sviluppo e nell'utilizzo del progetto rappresenta un aspetto fondamentale per assicurarne la buona riuscita. Per costruire un quadro accurato dei diversi profili utente di BugBoard26, si prevede la raccolta delle seguenti informazioni:

- **Generalità** : dati anagrafici di base, come nome, cognome, sesso ed età;
- **Breve biografia**: descrizione sintetica utile a inquadrare il contesto personale e professionale del soggetto;
- **obiettivi** : cosa la persona si aspetta di ottenere usando BugBoard26;
- **Skills** : abilità e conoscenze rilevanti per lo svolgimento delle attività lavorative;
- **Interessi** : aree tematiche o attività di particolare rilevanza per l'utente;
- **Motivazioni** : obiettivi e aspettative che spingono l'utente a interagire con il sistema;
- **Frustrazioni** : difficoltà o problematiche ricorrenti che possono influenzare negativamente l'esperienza d'uso.

La raccolta e l'analisi di tali caratteristiche consentiranno di supportare al meglio la progettazione e lo sviluppo della piattaforma, garantendo che tutti i soggetti coinvolti possano trarre beneficio dall'utilizzo di BugBoard26 e restare soddisfatti dell'esperienza complessiva.



### 2.2.1 Personas 1 : Marco Bianchi



#### Marco Bianchi

28 anni, uomo

amministratore di sistema

#### Interessi

Marco è fortemente interessato a strumenti che semplificano la gestione dei team preservando la sicurezza informatica. Ama sperimentare nuove soluzioni sul mondo del lavoro

#### Biografia

Amministratore di sistema con cinque anni di esperienza nel settore IT, Marco è abituato a gestire infrastrutture complesse e a garantire la sicurezza e l'efficienza dei servizi aziendali. È spesso responsabile della configurazione e della manutenzione di account e permessi utente, attività che richiedono precisione e affidabilità.

#### Skills

- **Gestione degli account e della sicurezza:** Esperto nella creazione e gestione di utenze, ruoli e permessi, con un'attenzione particolare alla protezione dei dati.
- **Amministrazione di sistemi:** Capacità di diagnosticare e risolvere problemi di accesso, configurazione e autenticazione.
- **Analisi e tracciamento delle modifiche:** Competente nell'utilizzo di log e cronologie per monitorare le attività degli utenti.

#### Motivazioni

Vuole un sistema che gli permetta di gestire rapidamente utenti e permessi, evitando passaggi complessi. È motivato dal poter garantire che ogni utente abbia accesso solo alle informazioni pertinenti e che tutte le modifiche siano tracciate in modo sicuro. Un sistema efficiente per la gestione delle utenze e la cronologia delle modifiche gli consente di lavorare con maggiore serenità.

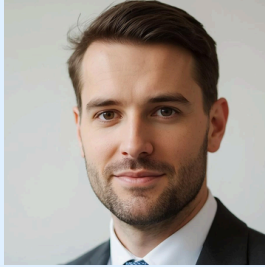
#### Frustazioni

Marco non gradisce quando i sistemi non evidenziano chiaramente la distinzione tra utenti e amministratori, creando confusione sui permessi di accesso e sulle responsabilità. Si infastidisce inoltre quando non può verificare rapidamente chi ha modificato un elemento o quando le informazioni sulle attività non sono tracciate.

Figura 2.1: Personas 1 : Marco Bianchi - amministratore di sistema



## 2.2.2 Personas 2 : Matteo Gentile



### Matteo Gentile

29 anni, uomo  
Senior Developer

#### Interessi

Matteo cerca strumenti che gli permettano di discutere direttamente nel contesto della issue, senza dover passare per email o chat esterne. Apprezza la possibilità di lasciare commenti per chiedere chiarimenti a chi ha aperto la segnalazione, proporre soluzioni alternative ad altri sviluppatori, o documentare le decisioni prese durante la risoluzione.

#### Motivazioni

È attratto dalla possibilità di utilizzare la sezione dei commenti per discutere di ciò su cui sta lavorando, poiché questo gli permette di evitare l'uso di canali esterni che rallenterebbero il suo lavoro. Vorrebbe inoltre poter visualizzare i commenti dei colleghi in ordine cronologico, così da mantenere sempre traccia della discussione ed essere aggiornato sugli sviluppi.

#### Biografia

Matteo lavora come senior developer e si occupa principalmente di risolvere issue complesse che richiedono analisi approfondite. Spesso ha bisogno di confrontarsi con chi ha segnalato il problema per ottenere chiarimenti o con altri sviluppatori per discutere possibili soluzioni. È abituato a lavorare su issue che richiedono iterazioni multiple prima di arrivare a una risoluzione definitiva.

#### Skills

- **Risoluzione di bug complessi:** Esperto nell'analizzare e risolvere problemi tecnici articolati che richiedono approfondimenti.
- **Collaborazione tecnica:** Abile nel comunicare con tester, altri sviluppatori e stakeholder per raccogliere informazioni necessarie.
- **Documentazione delle soluzioni:** Competente nel lasciare traccia delle decisioni tecniche prese durante la risoluzione.

#### Frustazioni

Matteo trova frustrante dover usare canali esterni per discutere di bug specifici, perché così perde il contesto e fatica a ritrovare le informazioni in seguito. Odia quando i commenti non sono organizzati cronologicamente o non è chiaro chi ha scritto cosa.

Figura 2.2: Personas 2 : Matteo Gentile - senior Developer



### 2.2.3 Personas 3 : Giulia Marino



#### Giulia Marino

**25 anni, donna**  
**Junior Developer**

##### Interessi

Le piace esplorare nuovi strumenti e tecnologie per lo sviluppo. Segue corsi online e tutorial per migliorare continuamente le sue competenze. Nel tempo libero partecipa a community di sviluppatori e legge articoli su come lavorare meglio in team e organizzare progetti software in modo efficace.

##### Motivazioni

Vuole crescere professionalmente e integrarsi meglio nel team di sviluppo, imparando a utilizzare in modo efficiente strumenti per gestire le issue assegnate e comprendere l'intero ciclo di risoluzione delle stesse.

##### Frustazioni

Si trova in difficoltà con piattaforme poco intuitive o con interfacce complesse. Si sente disorientata quando non riesce a capire chi ha modificato una segnalazione o in che stato si trova una issue da svolgere.

##### Biografia

Giulia ha 25 anni e ha da poco iniziato a lavorare come sviluppatrice junior in un piccolo team di sviluppo software.

Apprezza piattaforme semplici e ben organizzate, che le permettano di comprendere velocemente il flusso di lavoro del team e le responsabilità assegnate a ciascuno.

##### Skills

- **Buona conoscenza di linguaggi di programmazione** come Java e Python
- **Capacità di problem solving**

Figura 2.3: Personas 3 : Giulia Marino - Junior Developer



## 2.2.4 Personas 4 : Sara Conti



### Sara Conti

25 anni, donna  
tester QA

#### Interessi

È interessata a strumenti che le permettano di segnalare rapidamente bug in maniera dettagliata, con la possibilità di allegare screenshot per rendere più chiara la comunicazione. Ama tutto ciò che contribuisce a migliorare la tracciabilità delle segnalazioni e a rendere più nitida e comprensibile la comunicazione tecnica tra i membri del team.

#### Motivazioni

Vuole ridurre al minimo i tempi che intercorrono tra il rilevamento di un bug e la sua correzione, fornendo descrizioni precise, complete e facilmente comprensibili.

#### Biografia

Sara lavora come tester per un'azienda di sviluppo software e si occupa di individuare bug e anomalie prima del rilascio dei prodotti. Ha un approccio analitico e preciso, con grande attenzione ai dettagli e alla riproducibilità degli errori.

#### Skills

- **Testing e validazione:**  
Capacità di eseguire test sistematici e di descrivere con chiarezza le condizioni che producono un bug.
- **Documentazione:** Esperta nel redigere report di bug completi e coerenti.
- **Collaborazione cross-team:**  
Abituata a comunicare costantemente con sviluppatori e project manager.

#### Frustazioni

Sara trova frustranti i sistemi che non consentono di distinguere facilmente le issue per stato o priorità, costringendola a perdere tempo in ricerche manuali. Si irrita quando i commenti non sono visibili chiaramente, rendendo difficile la collaborazione con il resto del team. Odia dover utilizzare più strumenti separati per comunicare, preferendo un'unica piattaforma dove tutto sia integrato e aggiornato in tempo reale.

Figura 2.4: Personas 4 : Sara Conti - tester QA





## 2.2.5 Personas 5 : Elena Ferri



### Elena Ferri

34 anni, donna  
Senior QA Manager

#### Interessi

Elena cerca strumenti che le permettano di vedere la storia completa di ogni issue: chi l'ha creata, chi l'ha modificata, quando sono avvenuti i cambiamenti e cosa è stato modificato esattamente.

#### Biografia

Elena coordina il team di quality assurance da 5 anni e ha la responsabilità di garantire che tutti i bug vengano gestiti correttamente. Deve spesso verificare perché certi bug richiedono più tempo del previsto, chi li ha in carico e quali modifiche sono state apportate. È abituata a dover rispondere a domande del management sull'andamento delle risoluzioni e necessita di dati precisi e verificabili.

#### Skills

- **Audit e verifica:** Esperta nel controllare che i processi vengano seguiti correttamente e che ogni bug sia gestito in modo appropriato.
- **Analisi dei flussi di lavoro:** Abile nell'identificare colli di bottiglia o problemi nei processi analizzando la storia delle issue.
- **Accountability del team:** Competente nel garantire che ogni membro del team si assuma le proprie responsabilità.

#### Motivazioni

Vuole poter ricostruire rapidamente la storia di qualsiasi issue per capire cosa è successo e perché. È motivata dalla necessità di garantire trasparenza e accountability nel team: ogni membro deve essere responsabile delle proprie azioni e delle modifiche apportate.

#### Frustazioni

Elena trova frustrante quando non riesce a capire chi ha modificato una issue o quando le informazioni sui cambiamenti sono incomplete o mancanti. Odia dover chiedere manualmente ai membri del team cosa hanno fatto perché il sistema non traccia le modifiche. La irrita non poter vedere timestamp precisi o dettagli esatti delle modifiche, rendendo impossibile ricostruire con certezza la sequenza degli eventi.

Figura 2.5: Personas 4 : Elena Ferri - Senior QA Manager



## 2.2.6 Analisi delle personas

Procediamo con l'analizzare i vari profili riportati precedentemente differenziandoli per categorie.

### Amministratori

#### Marco Bianchi - Amministratore di sistema

Marco ricopre il ruolo di amministratore di sistema e, come si può facilmente intuire, nutre un forte interesse per il mondo della sicurezza informatica. Per questo motivo, il suo ruolo non si limita a quello di un semplice tester o sviluppatore, ma comprende una serie di responsabilità più ampie e complesse.

Tra i compiti principali di un amministratore rientra la possibilità di nominare nuovi utenti o amministratori, e Marco desidera che il sistema permetta di identificare in modo chiaro e immediato le azioni riservate agli amministratori rispetto a quelle accessibili agli utenti comuni. Avere una distinzione chiara delle funzioni e dei permessi è per lui fondamentale per lavorare in modo efficace e sicuro.

Allo stesso tempo, Marco non gradisce quando si trova a usare un sistema poco intuitivo o che non fornisce informazioni essenziali che un amministratore dovrebbe conoscere, come ad esempio una cronologia delle modifiche per le issue. Un'interfaccia chiara e trasparente non solo semplifica il suo lavoro quotidiano, ma gli permette anche di mantenere il controllo e la sicurezza dell'intero sistema in maniera efficiente.

### Sviluppatori

Dalle personas degli sviluppatori emerge chiaramente la necessità di strumenti intuitivi e ben organizzati che facilitino sia la segnalazione di problemi che la comprensione del flusso di lavoro del team. Entrambe le figure che ricoprono questo ruolo manifestano l'esigenza di interfacce chiare e processi semplificati, pur con diversi livelli di esperienza.

#### Giulia Marino - Junior Developer

Giulia rappresenta una sviluppatrice che da poco si occupa della risoluzione di issue riguardanti progetti software. Essendo all'inizio della sua carriera professionale, si trova nella fase delicata di dover apprendere contemporaneamente sia le dinamiche del lavoro in team che l'utilizzo di strumenti complessi di gestione. Per questo motivo ha bisogno di perdere il minor tempo possibile nell'imparare a usare la piattaforma, in modo da poter concentrare le sue energie sull'ambientamento nel nuovo ruolo e sulla comprensione delle logiche di sviluppo del team.

La sua biografia evidenzia come apprezzi piattaforme semplici e ben organizzate che non aggiungano ulteriore complessità a un contesto già impegnativo per chi è alle prime esperienze. Contestualmente, Giulia spera che l'utilizzo costante di questa applicazione le permetta di acquisire progressivamente familiarità con l'intero processo di risoluzione dei problemi, dalla segnalazione iniziale fino alla chiusura definitiva della issue.

Dalle sue motivazioni emerge inoltre che desidera avere una visione immediata e intuitiva di quali issue le sono state assegnate, a che punto del processo di risoluzione si trovano e quale priorità hanno. Questa chiarezza le è fondamentale per poter organizzare efficacemente il proprio lavoro quotidiano.

#### Matteo Gentile - Senior Developer

A differenza di Giulia, Matteo rappresenta lo sviluppatore esperto, il cui compito principale è risolvere issue complesse che richiedono analisi approfondite e collaborazione. La sua esperienza lo porta a valorizzare non tanto la semplicità dell'interfaccia, quanto l'efficienza della comunicazione all'interno



del team.

La sua frustrazione principale è quella di dover usare canali terzi per comunicare , questo ci suggerisce che l'implementazione di una sezione commenti è cruciale poiché negli stessi si può comunicare di una strategia di risoluzione o in generale dare aggiornamenti.

## **Team di Quality Assurance (QA)**

### **Sara Conti - Tester QA**

Sara è la figura operativa del team QA. Il suo compito è individuare, documentare e segnalare i bug. Per lei è fondamentale disporre di uno strumento che le consenta di creare issue dettagliate e inequivocabili. La sua frustrazione nasce da sistemi che limitano le informazioni che può fornire. Di conseguenza, per Sara è essenziale poter non solo definire un titolo e una descrizione, ma anche assegnare una priorità (per indicare l'urgenza), una tipologia (per categorizzare il problema) e, soprattutto, poter allegare immagini. Questi elementi sono vitali per dimostrare l'anomalia e fornire agli sviluppatori tutte le informazioni necessarie per riprodurla e risolverla rapidamente, riducendo al minimo i passaggi intermedi di comunicazione.

### **Elena Ferri - Senior QA Manager**

Elena ricopre un ruolo di supervisione e ha la responsabilità di garantire la qualità del processo, non solo del prodotto. La sua esigenza principale è la trasparenza e la tracciabilità. Non le basta sapere che un bug è stato risolto, ma deve poter ricostruire l'intera storia di una issue: chi l'ha creata, chi l'ha modificata, quando e quali cambiamenti sono stati apportati. La sua frustrazione più grande è l'assenza di informazioni sui cambiamenti o la presenza di dati incompleti, che le impediscono di identificare colli di bottiglia o problemi nel flusso di lavoro. Per Elena, quindi, l'implementazione di una cronologia delle modifiche per ogni issue è un requisito non negoziabile.



## 2.3 Casi d'uso - Use Case Diagram

Dopo aver analizzato i profili utente e le loro esigenze, procediamo con la definizione dei casi d'uso del sistema.

I diagrammi dei casi d'uso illustrano le interazioni tra gli attori e il sistema, evidenziando le funzionalità principali che la piattaforma deve offrire. Questa rappresentazione permette di visualizzare in modo chiaro e sintetico quali azioni possono essere compiute dai diversi ruoli e come questi interagiscono con il sistema per raggiungere i propri obiettivi.

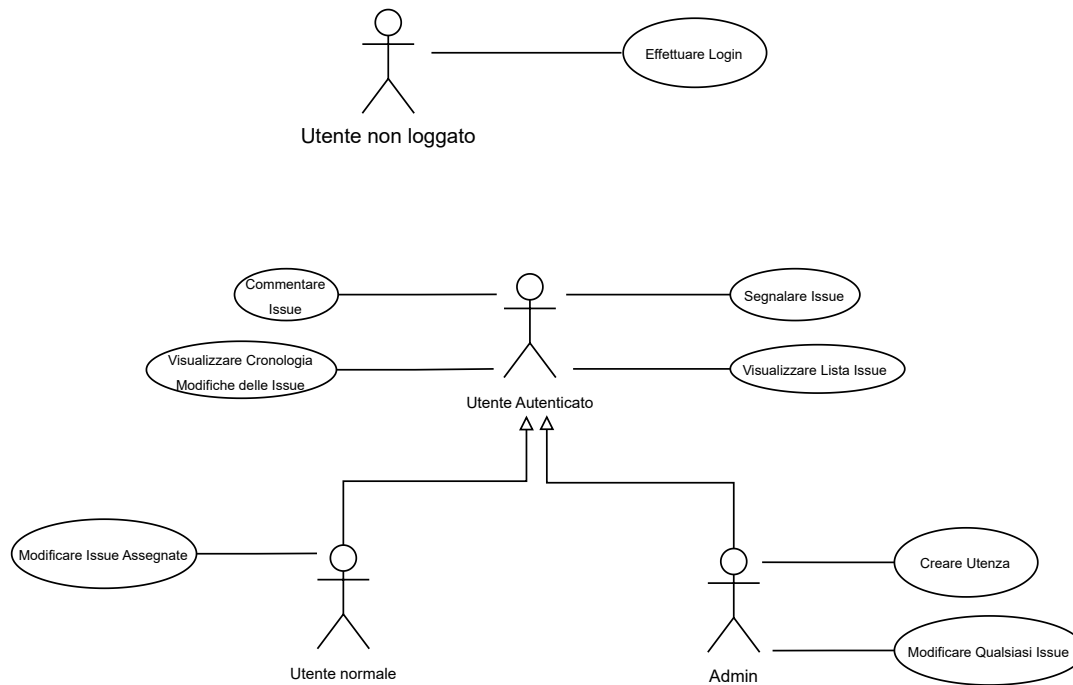


Figura 2.6: Use case BugBoard26

### 2.3.1 Analisi del Diagramma dei Casi d'Uso

La figura mostrata in precedenza rispecchia in maniera sintetica e schematica quanto analizzato nei paragrafi precedenti. Procediamo ora commentando gli attori elencati e i relativi casi d'uso.

#### Utente Non Loggato

Rappresenta qualsiasi utente che si interfaccia con l'applicazione senza aver prima eseguito l'accesso. Di conseguenza, l'unica operazione che può svolgere è **Effettuare Login**.



## Utente Autenticato

Rappresenta la generalizzazione degli attori principali del sistema (Amministratore e Utente Normale). Questo attore astratto raggruppa tutte le operazioni che un utente può effettuare dopo aver eseguito il login, indipendentemente dal suo ruolo specifico. Le funzionalità comuni sono:

- **Segnalare Issue**
- **Visualizzare Lista Issue**
- **Commentare Issue**
- **Visualizzare Cronologia Modifiche delle Issue**

## Utente Normale

Rappresenta un membro standard del team di sviluppo. Questo attore eredita tutte le funzionalità di Utente Autenticato e aggiunge la capacità di **Modificare Issue**, limitata alle sole issue che gli sono state esplicitamente assegnate.

## Amministratore

Rappresenta l'utente con i privilegi più elevati del sistema. Oltre a poter eseguire tutte le operazioni di un Utente Autenticato, l'Amministratore può:

- **Creare nuove utenze**
- **Modificare qualsiasi issue**



## 2.4 Mock-up a bassa fedeltà

In questa sezione andremo a rappresentare graficamente una funzionalità del sistema. Obiettivo di tale rappresentazione è fornire una prima idea intuitiva di come l'utente si interfacerà col nostro sistema.

### Mock-up caso d'uso inserimento nuova issue

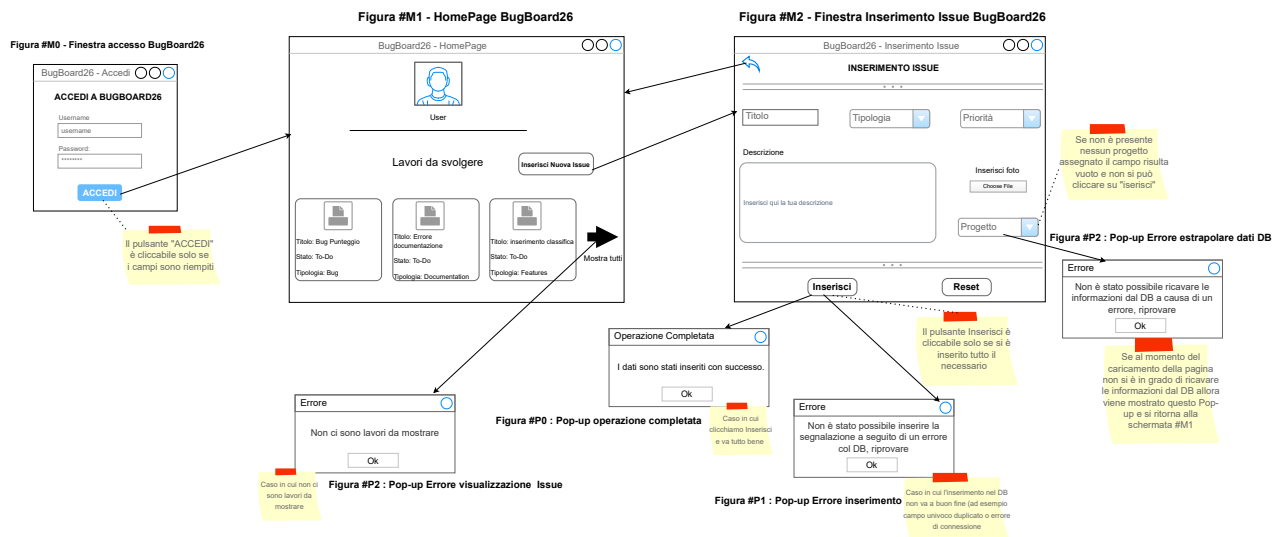


Figura 2.7: Mock-up caso d'uso Inserimento Issue - BugBoard26



## 2.5 Descrizione requisiti funzionali - Tabelle di Cockburn

In questa sezione vengono descritti nel dettaglio i casi d'uso del sistema attraverso le Tabelle di Cockburn. Questa rappresentazione tabellare permette di specificare in modo chiaro e strutturato le interazioni tra gli attori e il sistema, definendo per ogni caso d'uso le precondizioni, il flusso principale di esecuzione e gli eventuali scenari alternativi.

### 2.5.1 Use Case 1 - Inserimento Nuova Issue

|                              |                                                             |
|------------------------------|-------------------------------------------------------------|
| <b>USE CASE 1</b>            | <b>Inserimento nuova issue</b>                              |
| <b>Goal in Context</b>       | Inserire una nuova Issue                                    |
| <b>Preconditions</b>         | L'utente deve essere loggato                                |
| <b>Success End Condition</b> | Salva la nuova segnalazione                                 |
| <b>Failed End Condition</b>  | Non salva nulla                                             |
| <b>Primary Actor</b>         | Utente Autenticato                                          |
| <b>Trigger</b>               | L'utente clicca sul pulsante "Inserisci nuova Issue" in #M1 |

Tabella 2.1: Tabella di Cockburn - Use Case 1

#### Main Scenario:

| Step | Utente Autenticato                                | System                                     |
|------|---------------------------------------------------|--------------------------------------------|
| 1    | L'utente preme il pulsante per inserire una issue |                                            |
| 2    |                                                   | Mostra la finestra per l'inserimento (#M2) |
| 3    | Compila il form e clicca su inserisci             |                                            |
| 4    |                                                   | Mostra messaggio corretto inserimento #P0  |
| 5    | Clicca "OK" e termina main scenario               |                                            |

Tabella 2.2: Tabella di Cockburn - Main scenario Use Case 1



#### Extension #1: Utente clicca su "Reset"

| Step | Utente Autenticato                                  | System                             |
|------|-----------------------------------------------------|------------------------------------|
| 3a   | Utente clicca sul pulsante "Reset"                  |                                    |
| 4a   |                                                     | Pulisce tutti i campi già inseriti |
| 5a   | Prosegue come nel main scenario dallo step 2 in poi |                                    |

Tabella 2.3: Tabella di Cockburn - Extension 1: Utente clicca pulsante "Reset" nella schermata #M2

#### Extension #2: Utente clicca sul pulsante indietro

| Step | Utente Autenticato                    | System                 |
|------|---------------------------------------|------------------------|
| 3b   | L'utente clicca sul pulsante Indietro |                        |
| 4b   |                                       | Mostra la finestra #M1 |
| 5b   | Continua come nel main scenario       |                        |

Tabella 2.4: Tabella di Cockburn - Extension 2: Utente clicca "indietro" nella schermata #M2

#### Extension #3: Errore di salvataggio nel db

| Step | Utente Autenticato              | System                                                                                                                  |
|------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 4c   |                                 | Si verifica un errore durante il salvataggio nel database, resetta i campi e invita a riprovare mostrando il pop-up #P1 |
| 5c   | Riparte da step 3 main scenario |                                                                                                                         |

Tabella 2.5: Tabella di Cockburn - Extension 3: Gestione errore generico DB.

#### Extension #4: Errore recupero dati DB

| Step | Utente Autenticato              | System                                                                                   |
|------|---------------------------------|------------------------------------------------------------------------------------------|
| 2d   |                                 | Si verifica un errore durante il recupero dei dati dal DB e viene mostrato il pop-up #P2 |
| 3d   | L'utente clicca "OK"            |                                                                                          |
| 4d   |                                 | Mostra schermata #M1                                                                     |
| 5d   | Continua come nel main scenario |                                                                                          |

Tabella 2.6: Tabella di Cockburn - Extension 4: Gestione recupero dati nel DB.





## 2.6 Requisiti non funzionali e di sistema

Nella seguente sezione vengono indicati i requisiti non funzionali e di sistema emersi durante la fase di analisi.

### 2.6.1 Requisiti non funzionali

Sono stati individuati come requisiti non funzionali i seguenti:

#### **Sicurezza**

Uno dei primi requisiti non funzionali individuati è la sicurezza. Le informazioni dei profili aziendali rappresentano dati di cruciale importanza; per questo motivo andranno crittografate, in modo da renderle più robuste a eventuali attacchi da parte di malintenzionati.

#### **Usabilità**

L'interfaccia deve essere progettata in maniera tale da essere facilmente usabile dai fruitori. Almeno il 95% dei fruitori deve saper svolgere i task principali dell'applicativo (come login , inserimento issue , modifica) in totale autonomia e senza commettere errori.

#### **Prestazioni**

Il sistema deve offrire un'esperienza d'uso fluida e reattiva. Le operazioni comuni (login, visualizzazione issue, inserimento commenti) devono completarsi entro un tempo massimo di 5 secondi in condizioni normali di utilizzo. Il caricamento di grandi quantità di dati (es. lista completa delle issue) può richiedere fino a 10 secondi.

#### **Affidabilità**

Il sistema deve garantire un'affidabilità non inferiore al 99% su base mensile durante i giorni lavorativi (lunedì–venerdì), corrispondente a un tempo di inattività massimo di circa 5 ore e 17 minuti al mese. Negli altri giorni, l'affidabilità non deve essere inferiore al 97%, corrispondente a un tempo di inattività massimo di circa 5 ore e 46 minuti al mese.

#### **Manutenibilità**

Il sistema deve essere progettato in modo modulare e scalabile, così da consentire l'aggiunta di nuove funzionalità o modifiche future senza richiedere interventi significativi sul codice esistente. La struttura del codice dovrà favorire la riusabilità dei componenti e ridurre le dipendenze tra i moduli.



## 2.6.2 Requisiti di Dominio

Nella seguente Sezione vengono indicati i requisiti di dominio emersi durante la fase di analisi.

### Tipologie di profili

Sono previste due principali tipologie di profili nel sistema :

- **Utente** : può svolgere le task comuni richieste dal sistema e precedentemente elencate.
- **Admin** : può svolgere tutto ciò che fa un Utente e in più ha la possibilità di creare nuove utenze e modificare tutte le issue create.

### Struttura delle Issue

Ogni issue deve avere obbligatoriamente un titolo, una descrizione e uno stato. Appena creata deve avere lo stato *TODO*. Le issue possono appartenere a una delle seguenti categorie: *Bug*, *Feature*, *Documentation* o *Question*.

### Stati delle Issue

Oltre allo stato iniziale "TODO", le issue possono assumere altri stati che ne indicano l'avanzamento nel ciclo di vita (ad esempio: "In Progress", "Resolved", "Closed"). La transizione tra gli stati deve essere tracciata nella cronologia delle modifiche.

### Commenti e collaborazione

Ogni issue deve disporre di una sezione commenti accessibile a tutti gli utenti autenticati.

### Cronologia delle modifiche

Ogni issue deve avere una propria cronologia che tenga traccia di tutti gli interventi effettuati su di essa, a partire dalla creazione fino alla chiusura.



## Capitolo 3

# System Design

Conclusa la parte relativa al Requirements Engineering, iniziamo questo nuovo capitolo dedicato al System Design. L'obiettivo di questa sezione è la redazione di un documento di natura più tecnica rispetto alla sezione precedente, che guiderà la successiva implementazione del sistema. Discuteremo in particolare l'architettura adottata, con l'obiettivo di decomporre il problema in sotto-problemi più piccoli. Questo approccio non solo agevola il lavoro parallelo all'interno del team, ma costituisce anche un buon presupposto per la futura scalabilità del sistema. Verranno inoltre analizzati i principali componenti del sistema, i moduli software e le interazioni tra le varie parti, in modo da fornire una visione chiara e strutturata dell'architettura complessiva.



## 3.1 Architettura

Iniziamo con lo specificare il tipo di architettura che il nostro sistema dovrà adottare, difatti come specificato dalla traccia useremo un'architettura *Client-Server*. Un'architettura client-server è un'architettura distribuita dove dati ed elaborazioni sono distribuiti sulla rete. Esistono due tipologie di nodi appartenenti alla rete che sono :

- Client : coloro che usufruiscono del servizio.
- Server : offrono servizi specifici.

La scelta di utilizzare un'architettura client-server oltre ad essere totalmente conforme con quanto richiesto dal committente ha anche molti vantaggi come ad esempio una buona scalabilità oltre che una buona sicurezza e buone prestazioni.

### 3.1.1 Descrizione dell'architettura

Una volta specificata la tipologia dell'architettura, possiamo scendere più in dettaglio indicando come è stata suddivisa internamente. Abbiamo infatti deciso di organizzare il sistema secondo una struttura multilivello (layered architecture), scelta che permette di separare in modo chiaro le responsabilità e rendere l'intero sistema più modulare e manutenibile. In particolare abbiamo deciso che la nostra architettura sia una N-layer ossia un'evoluzione della 3-layer.

Mentre l'architettura 3-layer classica suddivide il sistema in Presentazione, Logica Applicativa (Business) e Dati, la nostra architettura N-layer introduce ulteriori livelli di astrazione per migliorare la flessibilità, la manutenibilità e soprattutto la scalabilità. Questa granularità aggiuntiva ci permette di isolare meglio le responsabilità, specialmente in un contesto orientato ai servizi. La nostra architettura sarà quindi così stratificata:

- **Livello di Presentazione (Presentation Layer)** : Questo è lo strato più esterno, responsabile della gestione dell'interfaccia utente (UI) e dell'interazione con l'utente finale. Include le applicazioni client (siano esse web-based o app mobili) che consumano i servizi esposti dal sistema.
- **Livello API Gateway (API Gateway Layer)** : Questo livello agisce come un unico punto d'ingresso per tutte le richieste provenienti dal Livello di Presentazione. È fondamentale nel nostro design perché disaccoppia i client dai servizi di backend.
- **Livello di Business (Business Layer)** : Questo è il cuore pulsante dell'applicazione. Contiene tutta la logica di business e le regole applicative. Seguendo un approccio moderno, questo livello non è concepito come un singolo blocco monolitico, ma è implementato attraverso un'architettura orientata ai servizi. Ogni servizio è un'unità indipendente, focalizzata su una specifica capacità di business, che espone le sue funzionalità tramite API.
- **Livello Dati (Data Layer)** : È lo strato più interno, responsabile della persistenza dei dati. Include i database e gestisce l'accesso e la memorizzazione fisica delle informazioni.

Questa suddivisione in N livelli ci permette di ottenere un'elevata coesione (ogni livello si occupa di una sola responsabilità) e un basso accoppiamento (i livelli comunicano tramite interfacce ben definite e possono essere sviluppati, distribuiti e scalati in modo indipendente).

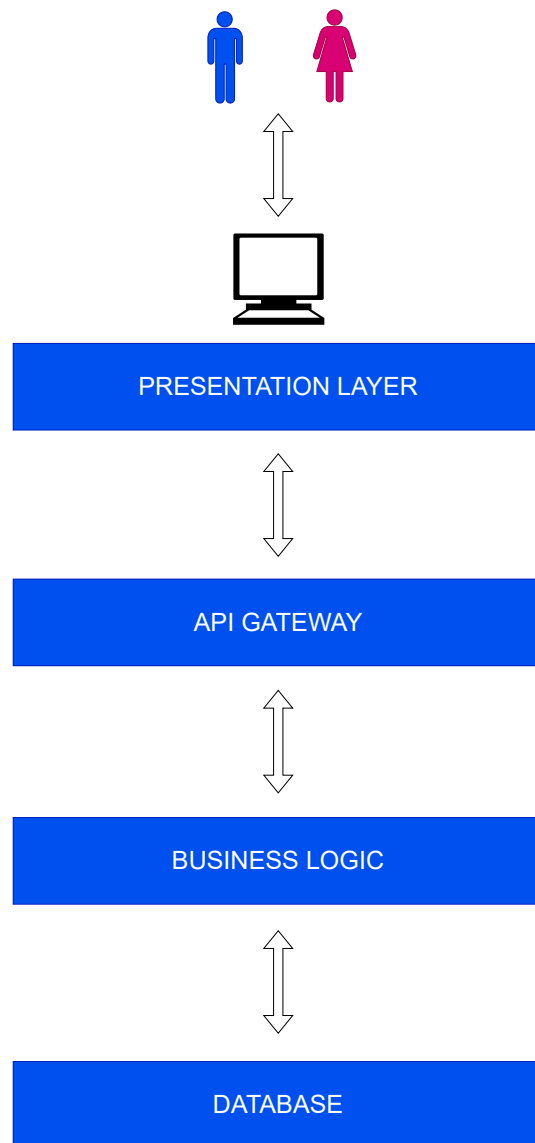


Figura 3.1: Descrizione ad alto livello dell'architettura



## Suddivisione in moduli

L'immagine mostrata in precedenza descrive la struttura generale del sistema a un livello alto di astrazione. È però utile soffermarsi maggiormente sulla business logic: a differenza di un'architettura monolitica, essa non sarà gestita da un singolo modulo centrale. Sebbene una struttura monolitica possa risultare più semplice da implementare inizialmente, essa rappresenterebbe un notevole ostacolo per le future estensioni del sistema.

Si consideri che il sistema in esame, essendo una piattaforma per la gestione di segnalazioni di issue, richiederà con buona probabilità l'introduzione periodica di nuove funzionalità. In un'architettura monolitica, con il passare del tempo, la gestione e l'evoluzione del codice diventerebbero sempre più difficili e poco manutenibili. Al contrario, suddividendo ogni funzionalità in moduli completamente indipendenti tra loro, l'aggiunta di nuove funzionalità diventa più semplice e sicura. Questa scelta comporta certamente una maggiore complessità organizzativa iniziale, ma garantisce flessibilità e scalabilità nel lungo periodo. Di seguito è riportata un'immagine che illustra in dettaglio lo schema dei moduli.

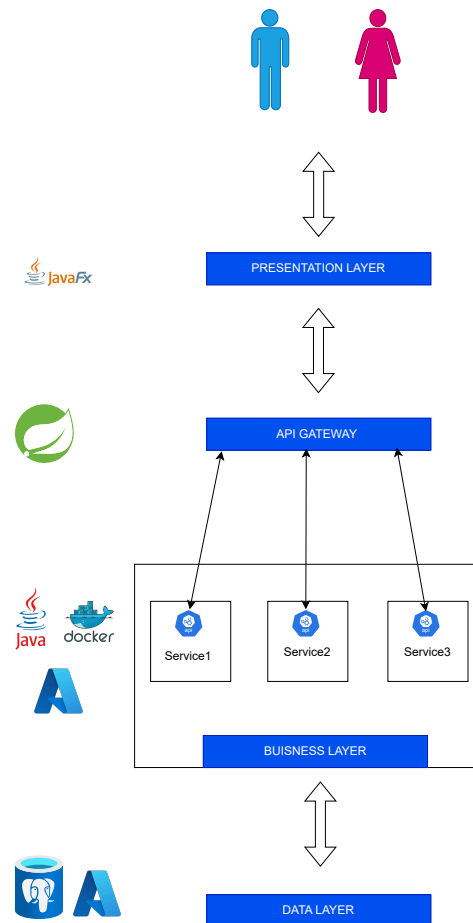


Figura 3.2: Descrizione nel dettaglio dell'architettura



## 3.2 Commento e descrizione delle tecnologie

Passiamo dunque a descrivere le varie tecnologie che useremo per ogni livello :

1. **Presentation Layer** : Il livello di presentazione sarà sviluppato utilizzando *JavaFX*, tecnologia che consente di realizzare interfacce grafiche moderne, reattive e facilmente estendibili. *JavaFX* risulta particolarmente adatto per applicazioni desktop e permette un buon isolamento tra logica della UI e logica applicativa.
2. **API Gateway** : Il punto di accesso unico al sistema sarà costituito da un insieme di *API REST*. Queste API fungono da canale di comunicazione standardizzato tra il frontend e i servizi di backend, garantendo un'interazione semplice, scalabile e indipendente dalla tecnologia utilizzata nei vari moduli.
3. **Business Layer** : La logica di business sarà implementata in *Java*, una scelta motivata dalla sua maturità, ampia diffusione e robustezza. Inoltre il gruppo possiede già una buona conoscenza del linguaggio, riducendo i tempi di apprendimento e aumentando l'affidabilità dello sviluppo. Per garantire l'indipendenza e l'isolamento dei diversi microservizi, ogni servizio verrà eseguito all'interno di un container *Docker*.
4. **Data Layer** : Per la gestione della persistenza dei dati verrà utilizzato il database relazionale *PostgreSQL*. Questa tecnologia è riconosciuta per la sua stabilità, performance e pieno supporto a vincoli, transazioni e operazioni avanzate sui dati.

## 3.3 Persistenza dei Dati

In questa sezione descriviamo nel dettaglio il livello dedicato alla persistenza dei dati, primo strato dell'architettura multilivello definita nella sezione precedente. Tale layer ha il compito di garantire una gestione strutturata, coerente e affidabile di tutte le informazioni utilizzate dal sistema, indipendentemente dai moduli applicativi che ne faranno uso. Per comprendere al meglio le entità principali presenti nel dominio applicativo e le loro relazioni, introduciamo un primo modello concettuale rappresentato tramite Class Diagram.



### 3.3.1 Class Diagram non ristrutturato

La figura seguente mostra una prima rappresentazione ad alto livello delle classi individuate durante l'analisi.

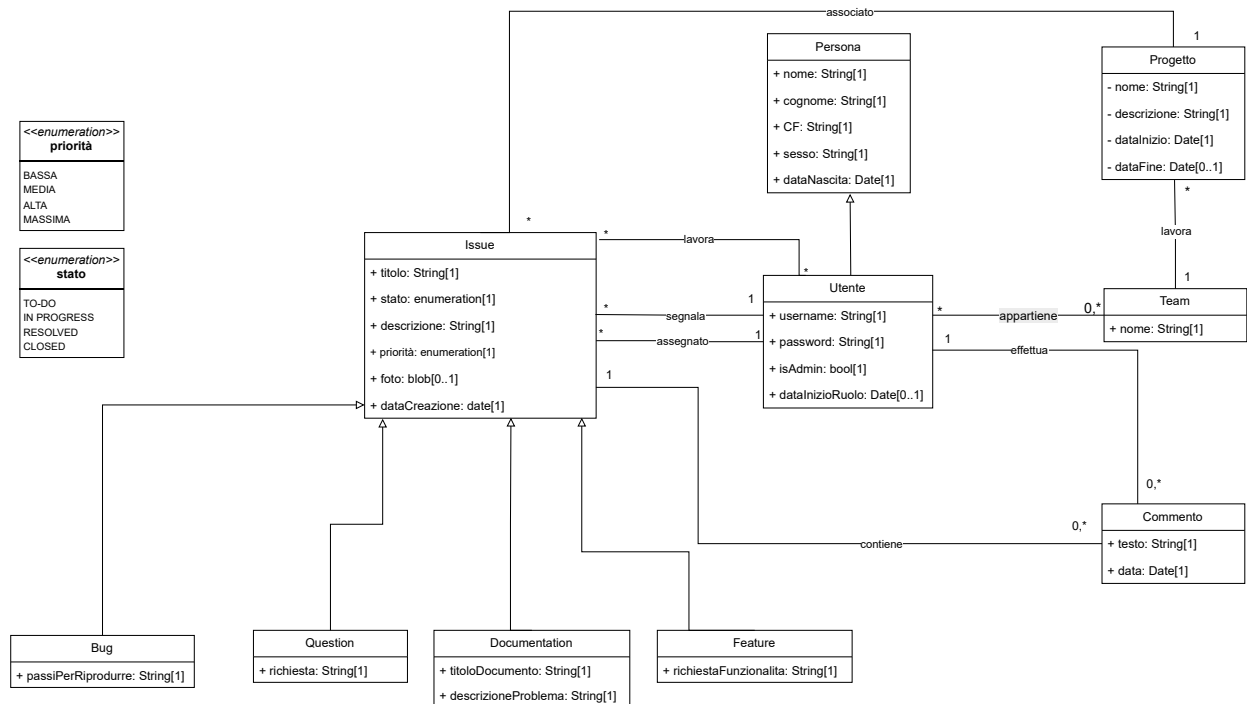


Figura 3.3: Class Diagram non ristrutturato





### 3.3.2 Class Diagram ristrutturato

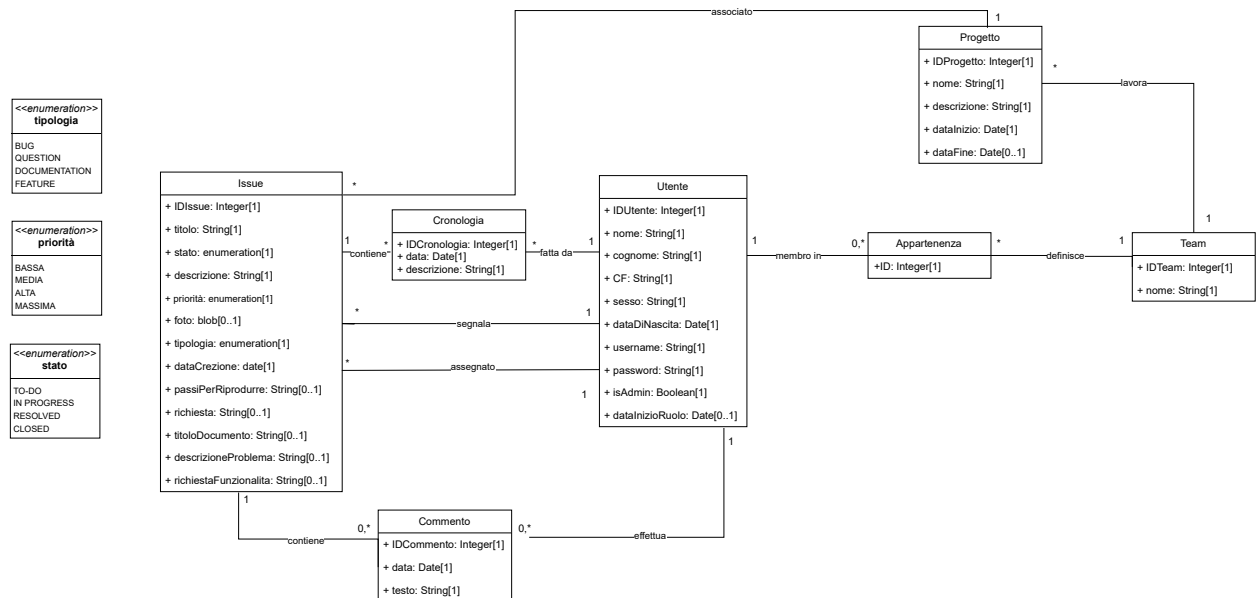


Figura 3.4: Class Diagram ristrutturato



### 3.3.3 Modello Logico

Dal class diagram ristrutturato possiamo scrivere questo schema logico

- Gli attributi sottolineati sono chiavi primarie.
- Gli attributi doppiamente sottolineati sono chiavi esterne.
- Gli attributi con \* possono essere NULL

**Utente** (IdUtente, Nome, Cognome, CF, Sesso, mail, DataDiNascita, Username, Password, IsAdmin, DataInizioRuolo\*)

---

**Team** (IdTeam, Nome)

---

**Appartenenza** (Id, IdUtente, IdTeam)

---

**Progetto** (IdProgetto, Nome, Descrizione, DataInizio, DataFine\*, IdTeam)

---

**Issue** (IdIssue, Titolo, Descrizione, Stato, Priorità, Foto\*, Tipologia, DataCreazione, PassiPerRiprodurre\*, Richiesta\*, TitoloDocumento\*, DescrizioneProblema\*, RichiestaFunzionalità\*, IdProgetto, IdSegnalatore, IdAssegnatario)

---

**Cronologia** (IdCronologia, Data, Descrizione\*, IdUtente, IdIssue)

---

**Commento** (IdCommento, Testo, Data, IdIssue, IdUtente)

---

Si precisa che IdSegnalatore e IdAssegnatario sono alias di IDUtente.



### 3.3.4 Modello fisico

Il modello fisico rappresenta l'implementazione concreta del modello logico descritto precedentemente, traducendo le entità e le relazioni in strutture dati effettive del database PostgreSQL.

Questa sezione presenta lo schema completo del database, includendo la definizione dei tipi enumerativi personalizzati, la creazione delle tabelle con i relativi vincoli di integrità referenziale e le regole di validazione dei dati.

Lo schema implementa tutti i requisiti emersi dall'analisi, garantendo:

- **Integrità dei dati:** attraverso vincoli di chiave primaria, chiave esterna e vincoli di dominio (CHECK constraints)
- **Tracciabilità completa:** mediante l'uso di politiche ON DELETE RESTRICT per preservare lo storico delle modifiche e delle segnalazioni
- **Coerenza semantica:** tramite l'utilizzo di tipi enumerativi per stato, priorità e tipologia delle issue
- **Prevenzione di duplicati:** attraverso vincoli di unicità su combinazioni di attributi critici

Di seguito viene riportato lo schema SQL completo per la creazione del database:

```
1 CREATE EXTENSION IF NOT EXISTS pgcrypto;
2 DROP TABLE IF EXISTS Commento CASCADE;
3 DROP TABLE IF EXISTS Cronologia CASCADE;
4 DROP TABLE IF EXISTS Issue CASCADE;
5 DROP TABLE IF EXISTS Progetto CASCADE;
6 DROP TABLE IF EXISTS Appartenenza CASCADE;
7
8
9 DROP TABLE IF EXISTS Utente CASCADE;
10 DROP TABLE IF EXISTS Team CASCADE;
11
12
13 DROP TYPE IF EXISTS enum_stato CASCADE;
14 DROP TYPE IF EXISTS enum_priorita CASCADE;
15 DROP TYPE IF EXISTS enum_tipologia CASCADE;
16
17 CREATE TYPE enum_stato AS ENUM (
18     'TO-DO',
19     'IN PROGRESS',
20     'RESOLVED',
21     'CLOSED'
22 );
23
24 CREATE TYPE enum_priorita AS ENUM (
25     'BASSA',
26     'MEDIA',
27     'ALTA',
28     'MASSIMA'
29 );
30
31 CREATE TYPE enum_tipologia AS ENUM (
32     'BUG',
33     'QUESTION',
34     'DOCUMENTATION',
35     'FEATURE'
36 );
```



```
37
38
39 CREATE TABLE Utente (
40     IdUtente SERIAL PRIMARY KEY,
41     Nome VARCHAR(100) NOT NULL,
42     Cognome VARCHAR(100) NOT NULL,
43     CF VARCHAR(16) NOT NULL UNIQUE,
44     DataDiNascita DATE NOT NULL,
45     Sesso CHAR(1) NOT NULL CHECK (Sesso IN ('M', 'F')),
46     Mail VARCHAR(50) NOT NULL UNIQUE,
47     Username VARCHAR(255) NOT NULL UNIQUE,
48     Password VARCHAR(255) NOT NULL,
49     IsAdmin BOOLEAN NOT NULL DEFAULT FALSE,
50     DataInizioRuolo DATE,
51
52     CONSTRAINT chk_admin_data CHECK ( (IsAdmin = TRUE AND DataInizioRuolo IS
53         NOT NULL) OR (IsAdmin = FALSE AND DataInizioRuolo IS NULL)),
54     CONSTRAINT check_lunghezza_CF CHECK (LENGTH(CF) = 16)
55 );
56
57
58 CREATE TABLE Team (
59     IdTeam SERIAL PRIMARY KEY,
60     Nome VARCHAR(100) NOT NULL UNIQUE
61 );
62
63
64 CREATE TABLE Appartenenza (
65     Id SERIAL PRIMARY KEY,
66     IdUtente INTEGER NOT NULL,
67     IdTeam INTEGER NOT NULL,
68
69     CONSTRAINT fk_appartenenza_utente FOREIGN KEY (IdUtente) REFERENCES Utente
70         (IdUtente) ON DELETE CASCADE,
71     CONSTRAINT fk_appartenenza_team FOREIGN KEY (IdTeam) REFERENCES Team(
72         IdTeam) ON DELETE CASCADE,
73     CONSTRAINT unica_appartenenza_utente_team UNIQUE (IdUtente, IdTeam)
74 );
75
76
77 CREATE TABLE Progetto (
78     IdProgetto SERIAL PRIMARY KEY,
79     Nome VARCHAR(200) NOT NULL,
80     Descrizione TEXT NOT NULL,
81     DataInizio DATE NOT NULL,
82     DataFine DATE,
83     IdTeam INTEGER NOT NULL,
84
85     CONSTRAINT fk_progetto_team FOREIGN KEY (IdTeam) REFERENCES Team(IdTeam)
86         ON DELETE RESTRICT,
87     CONSTRAINT chk_progetto_date CHECK (DataFine IS NULL OR DataFine >=
88         DataInizio)
89 );
```



```
88 CREATE TABLE Issue (
89     IdIssue SERIAL PRIMARY KEY,
90     Titolo VARCHAR(200) NOT NULL,
91     Descrizione TEXT NOT NULL,
92     Stato enum_stato NOT NULL DEFAULT 'TO-DO',
93     Priorita NOT NULL enum_priorita,
94     Foto BYTEA,
95     Tipologia enum_tipologia NOT NULL,
96     DataCreazione TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
97     IdProgetto INTEGER NOT NULL,
98     IdSegnalatore INTEGER NOT NULL,
99     IdAssegnatario INTEGER,
100     PassiPerRiprodurre TEXT,
101     Richiesta TEXT,
102     TitoloDocumento TEXT,
103     DescrizioneProblema TEXT,
104     RichiestaFunzionalita TEXT,
105
106     CONSTRAINT fk_issue_progetto FOREIGN KEY (IdProgetto) REFERENCES Progetto(
107         IdProgetto) ON DELETE CASCADE,
108     CONSTRAINT fk_issue_segnalatore FOREIGN KEY (IdSegnalatore) REFERENCES
109         Utente(IdUtente) ON DELETE RESTRICT,
110     CONSTRAINT fk_issue_assegnatario FOREIGN KEY (IdAssegnatario) REFERENCES
111         Utente(IdUtente) ON DELETE RESTRICT,
112
113     CONSTRAINT chk_bug_attributes CHECK (Tipologia != 'BUG' OR
114         PassiPerRiprodurre IS NOT NULL),
115     CONSTRAINT chk_question_attributes CHECK (Tipologia != 'QUESTION' OR
116         Richiesta IS NOT NULL),
117     CONSTRAINT chk_documentation_attributes CHECK (Tipologia != 'DOCUMENTATION
118         ' OR (TitoloDocumento IS NOT NULL AND DescrizioneProblema IS NOT NULL)
119         ),
120     CONSTRAINT chk_feature_attributes CHECK (Tipologia != 'FEATURE' OR
121         RichiestaFunzionalita IS NOT NULL)
122 );
123
124 CREATE TABLE Cronologia (
125     IdCronologia SERIAL PRIMARY KEY,
126     Data TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
127     Descrizione NOT NULL TEXT,
128     IdUtente INTEGER NOT NULL,
129     IdIssue INTEGER NOT NULL,
130
131     CONSTRAINT fk_cronologia_utente FOREIGN KEY (IdUtente) REFERENCES Utente(
132         IdUtente) ON DELETE RESTRICT,
133     CONSTRAINT fk_cronologia_issue FOREIGN KEY (IdIssue) REFERENCES Issue(
134         IdIssue) ON DELETE CASCADE
135 );
136
137 CREATE TABLE Commento (
138     IdCommento SERIAL PRIMARY KEY,
139     Testo TEXT NOT NULL,
140     Data TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
141     IdIssue INTEGER NOT NULL,
```



```
134     IdUtente INTEGER NOT NULL,
135
136     CONSTRAINT fk_commento_issue FOREIGN KEY (IdIssue) REFERENCES Issue(
137         IdIssue) ON DELETE CASCADE,
138     CONSTRAINT fk_commento_utente FOREIGN KEY (IdUtente) REFERENCES Utente(
139         IdUtente) ON DELETE RESTRICT
140 );
141
142
143
144 CREATE OR REPLACE FUNCTION check_assegnatario_nel_team()
145     RETURNS TRIGGER AS $$
146 DECLARE
147     team_progetto INTEGER;
148     is_membro BOOLEAN;
149 BEGIN
150     IF NEW.IdAssegnatario IS NULL THEN
151         RETURN NEW;
152     END IF;
153
154
155     SELECT IdTeam INTO team_progetto
156     FROM Progetto
157     WHERE IdProgetto = NEW.IdProgetto;
158
159
160     SELECT EXISTS (
161         SELECT 1
162         FROM Appartenenza
163         WHERE IdUtente = NEW.IdAssegnatario
164             AND IdTeam = team_progetto
165     ) INTO is_membro;
166
167
168     IF NOT is_membro THEN
169         RAISE EXCEPTION 'L''utente assegnatario (ID %) non fa parte del team
170             che gestisce questo progetto.', NEW.IdAssegnatario;
171     END IF;
172
173     RETURN NEW;
174 END;
175 $$ LANGUAGE plpgsql;
176
177 DROP TRIGGER IF EXISTS trg_check_assegnatario ON Issue;
178
179 CREATE TRIGGER trg_check_assegnatario
180     BEFORE INSERT OR UPDATE ON Issue
181     FOR EACH ROW
182     EXECUTE FUNCTION check_assegnatario_nel_team();
183
184
185
186 CREATE OR REPLACE FUNCTION prevent_change_closed_issue()
```



```
187 RETURNS TRIGGER AS $$
188 BEGIN
189     IF OLD.Stato = 'CLOSED' AND NEW.Stato = 'CLOSED' THEN
190         IF NEW.Titolo != OLD.Titolo OR
191            NEW.Descrizione != OLD.Descrizione OR
192            NEW.Priorita != OLD.Priorita THEN
193
194             RAISE EXCEPTION 'Non è possibile modificare i dettagli di una
195                               Issue nello stato CLOSED.';
196         END IF;
197     END IF;
198     RETURN NEW;
199 END;
200 $$ LANGUAGE plpgsql;
201
202 DROP TRIGGER IF EXISTS trg_freeze_closed_issue ON Issue;
203
204 CREATE TRIGGER trg_freeze_closed_issue
205     BEFORE UPDATE ON Issue
206     FOR EACH ROW
207 EXECUTE FUNCTION prevent_change_closed_issue();
208
209
210
211
212 CREATE OR REPLACE FUNCTION check_commento_su_issue_aperta()
213     RETURNS TRIGGER AS $$
214 DECLARE
215     stato_issue enum_stato;
216 BEGIN
217
218     SELECT Stato INTO stato_issue
219     FROM Issue
220     WHERE IdIssue = NEW.IdIssue;
221
222
223     IF stato_issue = 'CLOSED' THEN
224         RAISE EXCEPTION 'Non puoi commentare una Issue chiusa (ID %).', NEW.
225             IdIssue;
226     END IF;
227
228     RETURN NEW;
229 END;
230 $$ LANGUAGE plpgsql;
231
232 DROP TRIGGER IF EXISTS trg_block_comments_closed ON Commento;
233
234 CREATE TRIGGER trg_block_comments_closed
235     BEFORE INSERT ON Commento
236     FOR EACH ROW
237 EXECUTE FUNCTION check_commento_su_issue_aperta();
238
239
240 CREATE OR REPLACE FUNCTION check_permessi_commento()
241     RETURNS TRIGGER AS $$
```



```
241 DECLARE
242     team_progetto INTEGER;
243     is_admin BOOLEAN;
244     is_membro_team BOOLEAN;
245 BEGIN
246
247     SELECT IsAdmin INTO is_admin
248     FROM Utente
249     WHERE IdUtente = NEW.IdUtente;
250
251     IF is_admin IS TRUE THEN
252         RETURN NEW;
253     END IF;
254
255     SELECT P.IdTeam INTO team_progetto
256     FROM Issue I
257         JOIN Progetto P ON I.IdProgetto = P.IdProgetto
258     WHERE I.IdIssue = NEW.IdIssue;
259
260
261     SELECT EXISTS (
262         SELECT 1
263         FROM Appartenenza
264         WHERE IdUtente = NEW.IdUtente
265             AND IdTeam = team_progetto
266     ) INTO is_membro_team;
267
268
269     IF NOT is_membro_team THEN
270         RAISE EXCEPTION 'Accesso negato: Solo i membri del team (o Admin)
271             possono commentare questo progetto.';
272     END IF;
273
274     RETURN NEW;
275 END;
276 $$ LANGUAGE plpgsql;
277
278 DROP TRIGGER IF EXISTS trg_check_permessi_commento ON Commento;
279
280 CREATE TRIGGER trg_check_permessi_commento
281 BEFORE INSERT ON Commento
282 FOR EACH ROW
283 EXECUTE FUNCTION check_permessi_commento();
284
285
286
287
288 CREATE OR REPLACE FUNCTION assegna_carico_bilanciato()
289 RETURNS TRIGGER AS $$
290 DECLARE
291     target_team INTEGER;
292     scelto_utente INTEGER;
293 BEGIN
294
295     IF NEW.IdAssegnatario IS NOT NULL THEN
```





```
296         RETURN NEW;
297     END IF;
298
299
300     SELECT IdTeam INTO target_team
301     FROM Progetto
302     WHERE IdProgetto = NEW.IdProgetto;
303
304
305     SELECT A.IdUtente INTO scelto_utente
306     FROM Appartenenza A LEFT JOIN Issue I ON A.IdUtente = I.IdAssegnatario AND
307         I.Stato IN ('TO-DO', 'IN PROGRESS')
308     WHERE A.IdTeam = target_team
309     GROUP BY A.IdUtente
310     ORDER BY
311         COALESCE(SUM(
312             CASE
313                 WHEN I.Priorita = 'MASSIMA' THEN 10
314                 WHEN I.Priorita = 'ALTA' THEN 5
315                 WHEN I.Priorita = 'MEDIA' THEN 3
316                 WHEN I.Priorita = 'BASSA' THEN 1
317                 ELSE 0
318             END
319             ), 0) ASC,
320     RANDOM()
321     LIMIT 1;
322
323     IF scelto_utente IS NULL THEN
324         RAISE EXCEPTION 'Impossibile assegnare automaticamente: Il Team del
325             progetto è vuoto.';
326     END IF;
327
328     NEW.IdAssegnatario := scelto_utente;
329
330     RETURN NEW;
331 END;
332 $$ LANGUAGE plpgsql;
333
334
335
336 DROP TRIGGER IF EXISTS trg_load_balancer ON Issue;
337
338 CREATE TRIGGER trg_load_balancer
339     BEFORE INSERT ON Issue
340     FOR EACH ROW
341     EXECUTE FUNCTION assegna_carico_bilanciato();
342
343
344
345 CREATE OR REPLACE FUNCTION check_maggiorenne()
346     RETURNS TRIGGER AS $$
347 BEGIN
348     IF NEW.DataDiNascita >= (CURRENT_DATE - INTERVAL '18 years') THEN
349         RAISE EXCEPTION 'L''utente deve essere maggiorenne. Data di nascita
```



```

350         inserita: %', NEW.DataDiNascita;
351     END IF;
352     RETURN NEW;
353 END;
354 $$ LANGUAGE plpgsql;
355
356
357 DROP TRIGGER IF EXISTS trg_check_maggiorenne ON Utente;
358
359 CREATE TRIGGER trg_check_maggiorenne
360     BEFORE INSERT OR UPDATE ON Utente
361     FOR EACH ROW
362 EXECUTE FUNCTION check_maggiorenne();
363
364
365 CREATE OR REPLACE FUNCTION log_solo_assegnazione()
366 RETURNS TRIGGER AS $$
367 DECLARE
368     nome_utente VARCHAR;
369     cognome_utente VARCHAR;
370 BEGIN
371     IF NEW.IdAssegnatario IS NOT NULL AND
372        (TG_OP = 'INSERT' OR NEW.IdAssegnatario IS DISTINCT FROM OLD.
373         IdAssegnatario) THEN
374
375 SELECT Nome, Cognome INTO nome_utente, cognome_utente
376 FROM Utente
377 WHERE IdUtente = NEW.IdAssegnatario;
378
379 INSERT INTO Cronologia (Data, Descrizione, IdUtente, IdIssue)
380 VALUES (
381         CURRENT_TIMESTAMP,
382         'Issue assegnata a ' || nome_utente || ' ' ||
383         cognome_utente,
384         NEW.IdAssegnatario,
385         NEW.IdIssue
386     );
387
388 END IF;
389
390 RETURN NEW;
391 END;
392 $$ LANGUAGE plpgsql;
393
394 DROP TRIGGER IF EXISTS trg_log_assegnazione ON Issue;
395
396 CREATE TRIGGER trg_log_assegnazione
397 AFTER INSERT OR UPDATE ON Issue
398 FOR EACH ROW
399 EXECUTE FUNCTION log_solo_assegnazione();
```

Listing 3.1: Codice DDL del Database BugBoard



## 3.4 Business Layer

L'analisi prosegue con l'esame del nucleo centrale del progetto: la logica di Business. Questo layer funge da intermediario tra le richieste dell'utente e la persistenza dei dati, trasformando i requisiti funzionali in operazioni software concrete.

### 3.4.1 Stack Tecnologico

Per lo sviluppo di questo strato è stato adottato un ecosistema basato su **Java**, scelta dettata dalla robustezza del linguaggio e dalla profonda competenza del team di sviluppo. Il cuore tecnologico del progetto si poggia su pilastri fondamentali:

- **Spring Framework (Spring Boot):** Utilizzato come colonna vertebrale dell'intera applicazione. Spring gestisce il ciclo di vita dei componenti tramite la *Dependency Injection* e, grazie al modulo *Spring Web*, permette l'esposizione della logica di business attraverso API REST. Questo garantisce un'interfaccia di comunicazione standard, sicura e facilmente integrabile con diverse tipologie di client.
- **Java Persistence API (JPA) e Hibernate:** Per la gestione dell'interazione con la base dati. L'astrazione fornita da JPA consente di operare sulle entità del dominio come oggetti Java, automatizzando le operazioni di persistenza e slegando la logica applicativa dalle specifiche del database relazionale.
- **Docker:** Adottato per la containerizzazione dei servizi. Ogni componente del backend è impacchettato in un container isolato, garantendo coerenza tra l'ambiente di sviluppo e quello di produzione e facilitando il deployment dell'intera infrastruttura.
- **Project Lombok:** Una libreria fondamentale per mantenere il codice pulito e leggibile. Attraverso l'uso di annotazioni, Lombok elimina il cosiddetto *boilerplate code* (getter, setter, costruttori), permettendo agli sviluppatori di concentrarsi esclusivamente sulla logica di business.
- **Apache Maven:** Utilizzato come strumento di gestione e *build automation*. Maven permette di gestire in modo centralizzato tutte le dipendenze del progetto (librerie esterne) tramite il file `pom.xml`, garantendo che l'intero team lavori con le stesse versioni dei framework. Inoltre, standardizza il processo di compilazione, test e packaging del software.

### 3.4.2 Architettura e Definizione di Servizio

L'architettura adottata non è di tipo monolitico, bensì **Orientata ai Servizi (SOA)**. Sebbene la struttura presenti forti analogie con un'architettura a microservizi, abbiamo preferito definirla SOA in quanto il sistema condivide un'infrastruttura di persistenza di dati comune, pur mantenendo una rigorosa separazione logica dei domini. Per garantire l'accessibilità e la continuità del servizio, l'intero backend containerizzato è stato dispiegato su una **Virtual Machine (VM) ospitata sull'infrastruttura cloud di Microsoft Azure**. Questa configurazione permette all'applicazione client di interagire con un backend remoto sempre attivo, svincolando l'esecuzione dai singoli ambienti locali di sviluppo. Abbiamo inteso come servizio una qualsiasi serie di operazioni logicamente dipendenti tra loro, che cooperano per fornire un'utilità all'utilizzatore.



### 3.4.3 Definizione architetturali dei moduli

L'architettura interna dei moduli segue un approccio *multi-tier* rigoroso, volto a disaccoppiare la logica di persistenza dalla logica di presentazione. Ogni modulo implementa i seguenti componenti standard:

- **Controller (REST Layer):** Gestiscono le richieste HTTP in ingresso, fungendo da punto di accesso per le API REST. Si occupano della validazione dell'input e della serializzazione della risposta, delegando l'elaborazione al livello sottostante.
- **DTO (Data Transfer Objects):** Oggetti utilizzati esclusivamente per il trasporto dei dati tra il client e il server. Il loro utilizzo permette di non esporre direttamente le entità del database, garantendo sicurezza e flessibilità nel versionamento delle API.
- **Service Layer:** Contiene la logica di business (*business logic*) vera e propria. Questo strato orchestra le operazioni, applica le regole di dominio e gestisce le transazioni. Per favorire la riusabilità del codice ogni servizio è definito da un'interfaccia e dalla relativa implementazione concreta.
- **Repository Interface:** Definisce i contratti per l'accesso ai dati, sfruttando le astrazioni di JPA per interagire con il database senza legare il codice a query SQL specifiche.
- **Entity (Domain Model):** Classi annotate che rappresentano la mappatura relazionale (ORM) delle tabelle del database, costituendo il nucleo persistente del dominio.

## 3.5 Login-Service

Il modulo *Login-Service* costituisce il componente critico per la sicurezza del sistema, agendo come *gatekeeper* per l'accesso alle risorse protette. La sua responsabilità primaria è l'autenticazione degli utenti e il rilascio di token di sicurezza validi (JWT) per le successive interazioni con gli altri microservizi.

Dall'analisi del codice, emerge come il modulo sia stato progettato non solo per la sicurezza, ma anche per garantire un'elevata portabilità su diverse applicazioni.

### 3.5.1 Workflow di Autenticazione e Sicurezza

Il flusso operativo è stato strutturato per massimizzare robustezza e flessibilità:

- **Flessibilità in ingresso:** Il metodo `findByUsernameOrEmail` nel layer di persistenza permette l'autenticazione indifferente tramite username o email, migliorando l'esperienza utente.
- **Sicurezza delle Credenziali:** L'uso di `PasswordEncoder` con algoritmo *BCrypt* assicura che le password siano salvate esclusivamente sotto forma di hash, proteggendo i dati anche in caso di accesso non autorizzato al DB.

### 3.5.2 Architettura: Riusabilità e Portabilità

Un punto di forza distintivo di questo modulo è l'adozione di pattern architetturali che ne favoriscono il riutilizzo in contesti applicativi differenti:

- **Disaccoppiamento tramite Interfacce (Interface-Driven Design):** La logica di business non è esposta direttamente, ma è mediata dall'interfaccia `LoginService`. Questa astrazione rende il sistema estremamente modulare: il `Controller` dipende esclusivamente dal contratto definito dall'interfaccia, ignorando i dettagli implementativi. Ciò significa che in futuro o in altre applicazioni, l'implementazione concreta (`LoginServiceImpl`) potrebbe essere sostituita (ad esempio per integrare protocolli come LDAP o OAuth2) senza che il codice client debba subire alcuna modifica.



- **Estensibilità del Domain Model:** Anche il modello dati è stato stratificato per favorire il riuso:
  - La classe astratta **BasicUser** (*@MappedSuperclass*) incapsula esclusivamente il nucleo essenziale dell'identità (username e password). Essendo priva di legami con il dominio specifico, questa classe è portabile e riutilizzabile "as-is" in qualsiasi altra applicazione che richieda un sistema di login basilare.
  - La classe concreta **User** estende questo nucleo aggiungendo i campi specifici del progetto corrente (anagrafica, ruoli, email). Questo approccio dimostra come sia possibile adattare il sistema di sicurezza a domini diversi mantenendo inalterato il core della logica di autenticazione.

### 3.5.3 Struttura dei Componenti

Il design segue rigorosamente la *Separation of Concerns*:

- Il **Controller** agisce da puro intermediario REST, validando l'input e delegando le operazioni al Service Layer.
- I **DTO** (*RichiestaLogin*, *RispostaLogin*) isolano il modello di dominio dall'esterno, garantendo che le modifiche al database non impattino sui contratti API pubblici.

### 3.5.4 Diagramma UML LoginService

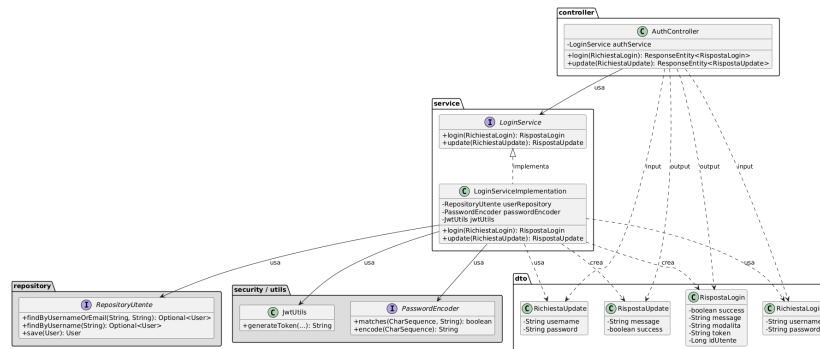


Figura 3.5: Diagramma UML servizio login



## 3.6 InserimentoUtente-Service

Il modulo *InserimentoUtente-Service* è responsabile della gestione del ciclo di vita della registrazione di nuovi account nel sistema. A differenza di una semplice operazione CRUD, questo servizio implementa una logica rigorosa di validazione e integrità dei dati, garantendo che solo le informazioni conformi alle regole di business vengano persistite.

Dall'analisi architetturale, emergono tre pilastri fondamentali che governano l'implementazione del modulo: la validazione preventiva, l'uso di pattern creazionali avanzati e la gestione centralizzata degli errori.

### 3.6.1 Validazione e Integrità dei Dati

La robustezza del servizio è garantita da un sistema di controllo a più livelli che previene l'inserimento di dati incoerenti o duplicati:

- **Validazione Dichiarativa (Bean Validation):** Invece di appesantire il codice con blocchi `if-else` per controllare l'input, il modulo sfrutta le annotazioni standard `Jakarta Validation` direttamente sul DTO `RichiestaInserimentoUtente`. Vincoli come `@NotBlank`, `@Size`, `@Pattern` (per il sesso o la password complessa) e `@Past` (per la data di nascita) definiscono i requisiti dei dati in modo dichiarativo e leggibile.
- **Gestione Centralizzata delle Eccezioni:** Il modulo introduce un componente architetturale chiave: il `GlobalExceptionHandler`. Questa classe, annotata con `@RestControllerAdvice`, intercetta automaticamente le eccezioni di validazione (`MethodArgumentNotValidException`) lanciate dal framework. Essa trasforma gli errori tecnici in risposte JSON strutturate e comprensibili per il client, contenenti la mappa dettagliata dei campi non validi. Questo approccio disaccoppia la logica di gestione degli errori dai Controller, mantenendo questi ultimi estremamente snelli.
- **Verifica di Unicità:** Prima di procedere alla persistenza, il Service Layer esegue controlli preventivi sul database per garantire l'unicità di campi critici quali *Username*, *Email* e *Codice Fiscale*, prevenendo conflitti e garantendo la consistenza del dominio.

### 3.6.2 Architettura e Design Pattern

Anche in questo modulo è stata mantenuta una forte attenzione alla riusabilità e alla pulizia del codice attraverso pattern specifici:

- **Portabilità del Modello (Inheritance):** Similmente al modulo di Login, l'entità segue una gerarchia ben definita. La classe base `Persona` (`@MappedSuperclass`) astrae i dati anagrafici comuni (nome, cognome, CF), rendendoli riutilizzabili per altre tipologie di attori nel sistema. La classe `Utente` estende `Persona` specializzandola con le credenziali di accesso e i ruoli, dimostrando un design orientato all'estensione.
- **Builder Pattern:** Per la costruzione dell'oggetto complesso `Utente`, è stato implementato il pattern *Builder*. Questo pattern creazionale risolve il problema dei costruttori con troppi parametri ("telescoping constructor antipattern"), permettendo una creazione dell'oggetto fluida, leggibile e meno soggetta a errori di scambio parametri.



### 3.6.3 Diagramma UML InserimentoUtenteService

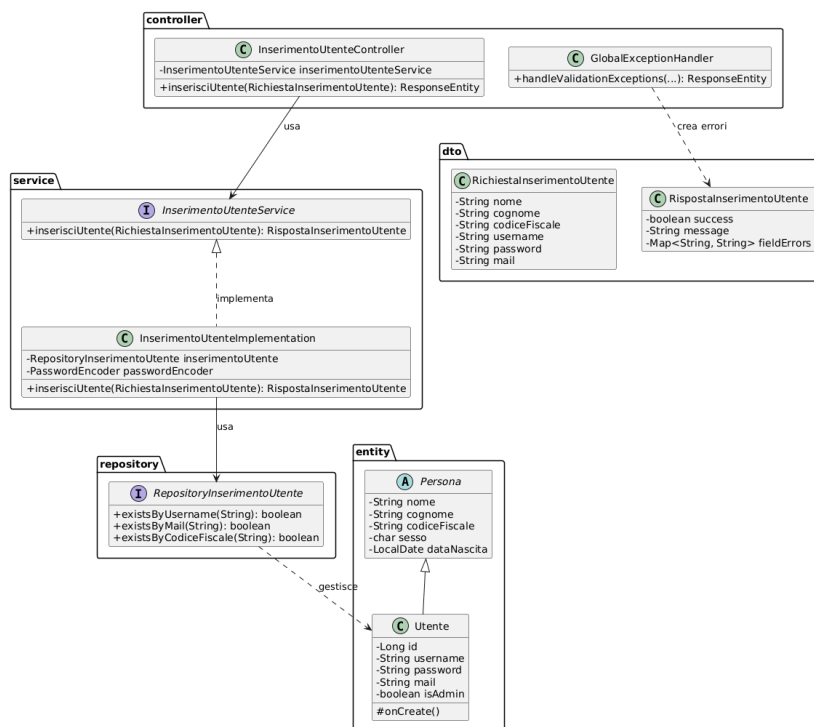


Figura 3.6: Diagramma UML servizio Inserimento Utente



## 3.7 InserimentoIssue-Service

Il modulo `InserimentoIssue-Service` è il componente del sistema *BugBoard* deputato alla creazione e alla persistenza delle segnalazioni (Issue). Questo microservizio gestisce un modello dati articolato, che comprende non solo dati testuali descrittivi, ma anche classificazioni tipologiche (priorità, tipologia) e dati binari (immagini allegate).

L'architettura del modulo è stata progettata per garantire la corretta mappatura tra i dati provenienti dal front-end e il modello di persistenza, assicurando coerenza e manutenibilità. Di seguito vengono analizzati i principali aspetti ingegneristici.

### 3.7.1 Modellazione del Dominio e Type Safety

A differenza di entità anagrafiche semplici, l'entità `Issue` rappresenta il core business dell'applicazione e richiede una classificazione rigorosa. Per garantire la coerenza dei dati e prevenire valori arbitrari, sono stati utilizzati dei tipi enumerativi (*Enums*) Java:

- **Tipologia:** L'uso dell'Enum `Tipologia` vincola la creazione della issue a categorie predefinite (`BUG`, `QUESTION`, `DOCUMENTATION`, `FEATURE`), facilitando successive operazioni di filtraggio e reporting.
- **Priorità:** Analogamente, l'Enum `Priorita` (`BASSA`, `MEDIA`, `ALTA`, `MASSIMA`) stabilisce una scala di gravità standardizzata, essenziale per i flussi di lavoro di assegnazione.

Questa scelta progettuale sposta il controllo di validità dal livello applicativo al livello di compilazione e di database (tramite l'annotazione `@Enumerated(EnumType.STRING)`), garantendo una maggiore robustezza del sistema.

### 3.7.2 Tracciabilità e Audit: L'Entità Cronologia

Un requisito non funzionale critico per un sistema di Bug Tracking è la capacità di ricostruire la storia delle modifiche. Il modulo include nel proprio dominio l'entità `Cronologia`, deputata a mantenere un *audit trail* delle operazioni.

Strutturalmente, questa entità funge da registro di collegamento, associando ogni evento a tre dimensioni fondamentali:

- **Temporale:** Tramite il campo `data` (`LocalDateTime`), che fissa l'istante esatto dell'operazione.
- **Operativa:** Attraverso una `descrizione` testuale dell'evento accaduto.
- **Relazionale:** Collegando l'evento sia all'attore che l'ha scatenato (`idUtente`) sia all'oggetto della segnalazione (`idIssue`).

La presenza del componente `CronologiaRepository` nel layer di persistenza evidenzia un'architettura predisposta non solo alla gestione dello stato corrente delle issue, ma anche alla storicizzazione del loro intero ciclo di vita, garantendo trasparenza e accountability nel processo di gestione dei bug.

### 3.7.3 Pattern Creazionali: Builder Pattern

L'entità `Issue` è caratterizzata da un elevato numero di attributi (titolo, descrizione, stato, ID referenziali, BLOB per foto, ecc.). L'utilizzo di un costruttore tradizionale o di molteplici metodi *setter* avrebbe reso il codice del Service Layer verboso e pronò ad errori (il cosiddetto *telescoping constructor antipattern*).

Per ovviare a questo problema, è stato adottato il **Builder Pattern** (tramite l'annotazione `@Builder` di Lombok). Nel metodo `insert` della classe `InserimentoIssueServiceImpl`, la creazione dell'oggetto avviene in modo dichiarativo e fluente:





```
Issue issue = Issue.builder()
    .titolo(richiesta.getTitolo())
    .descrizione(richiesta.getDescrizione())
    .priorita(richiesta.getPriorita())
    .foto(richiesta.getImmagine())
    // ... altri campi
    .build();
```

Questo approccio migliora drasticamente la leggibilità del codice e semplifica la manutenzione nel caso in cui il modello dati dovesse evolvere con nuovi campi.

### 3.7.4 Gestione della Transazionalità e Persistenza

Il servizio implementa una gestione esplicita delle transazioni tramite l'annotazione `@Transactional` sul metodo di inserimento. Sebbene l'operazione principale sia una singola `save` sul repository `InserimentoIssueRepository`, l'uso di un contesto transazionale garantisce che l'operazione sia atomica e isolata, preparandosi a future evoluzioni in cui l'inserimento di una `Issue` potrebbe dover essere coordinato con altre operazioni di scrittura (es. aggiornamento di statistiche o notifiche).

Il controller `InserimentoIssueController` funge da semplice facciata REST, delegando interamente la logica di business al service e mappando l'esito dell'operazione (incapsulato nel DTO `RispostaInserimentoIssue`) nei corretti HTTP Status Code (201 Created in caso di successo, 400 Bad Request in caso di errore).

### 3.7.5 Diagramma UML InserimentoIssueService

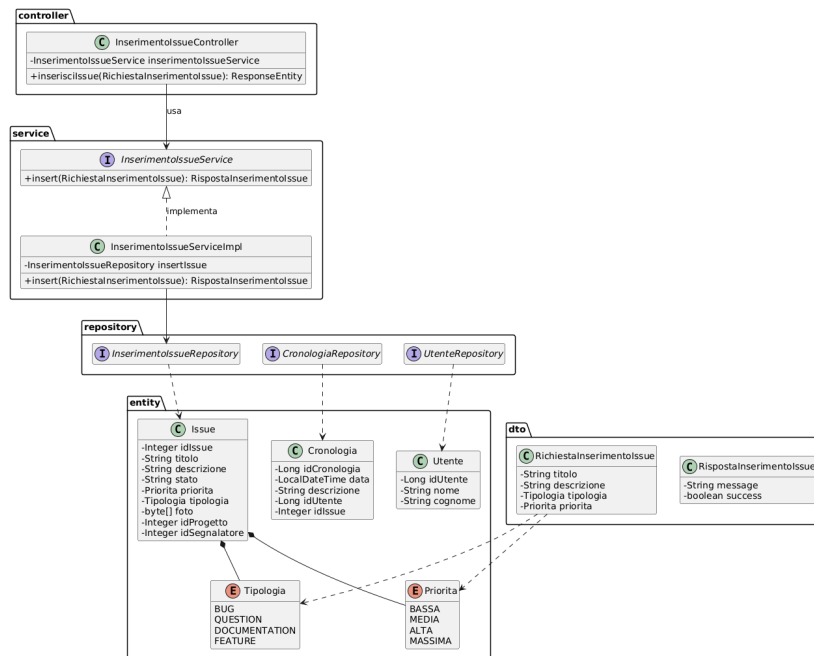


Figura 3.7: Diagramma UML servizio Inserimento Issue



## 3.8 RecuperoIssue-Service

Il modulo **RecuperoIssue-Service** rappresenta il componente di lettura principale del sistema, responsabile dell'interrogazione e della presentazione dei dati relativi alle segnalazioni. A differenza dei servizi di inserimento, caratterizzati da logiche di validazione e scrittura, questo microservizio affronta le sfide legate all'aggregazione delle informazioni e all'efficienza nel recupero di grandi moli di dati.

L'analisi del codice evidenzia tre pattern architetturali predominanti: l'uso del *Repository Pattern* per query complesse, l'implementazione della paginazione lato server e il pattern *Data Transfer Object (DTO) Projection* per l'appiattimento delle relazioni.

### 3.8.1 Ottimizzazione delle Query e Paginazione

Per garantire tempi di risposta costanti indipendentemente dalla crescita del database, il servizio implementa nativamente la paginazione tramite l'interfaccia **Pageable** di Spring Data JPA. Il **RecuperoIssueRepository** definisce metodi annotati con **@Query** personalizzate (JPQL) per filtrare le issue in base a criteri specifici escludendo sistematicamente quelle nello stato 'CLOSED':

```
@Query("SELECT DISTINCT i FROM Issue i WHERE i.idProgetto = :idProgetto  
AND i.stato != 'CLOSED'")  
Page<Issue> findByIdProgettoAndStatoNot(..., Pageable pageable);
```

Questo approccio delega al DBMS il carico del filtraggio e dell'ordinamento, riducendo drasticamente il traffico di rete e l'uso della memoria applicativa rispetto a un filtraggio in-memory.

### 3.8.2 Aggregazione e Composite View

Una delle funzionalità critiche del servizio è la ricostruzione della vista di dettaglio di una Issue (endpoint **recuperaDettaglio**). Un singolo oggetto **Issue** nel database è collegato logicamente a molteplici entità correlate: l'autore, l'assegnatario, la lista dei commenti e la cronologia delle modifiche.

Il **RecuperoIssueServiceImpl** agisce come un orchestratore che aggrega questi dati eterogenei in un unico oggetto di risposta (**RispostaDettaglioIssue**). Il servizio esegue query parallele mirate sui repository **CommentoRepository** e **CronologiaRepository**, ordinando i risultati cronologicamente, e popola i DTO risultanti risolvendo le chiavi esterne (ID Utente) nei rispettivi dati anagrafici (Nome e Cognome) per una fruizione immediata da parte del frontend.

### 3.8.3 Gestione delle Relazioni: Eager vs Lazy Loading

È interessante notare un approccio ibrido nella gestione delle relazioni JPA:

- **Issue:** Mantiene riferimenti "leggeri" agli utenti tramite ID numerici (es. `idAssegnatario`), richiedendo una lookup esplicita nel service per recuperare i dettagli dell'utente.
- **Commento e Cronologia:** Utilizzano relazioni JPA strutturate (`@ManyToOne(fetch = FetchType.LAZY)`) verso l'entità **Utente**.

Questa distinzione suggerisce un'ottimizzazione mirata: mentre la Issue è l'entità radice e viene caricata spesso in liste (dove la join con l'utente potrebbe essere costosa se non necessaria), i commenti e la cronologia sono caricati solo nel dettaglio, dove la presenza dell'autore è sempre richiesta, giustificando la relazione diretta.



### 3.8.4 Diagramma UML RecuperoIssueService

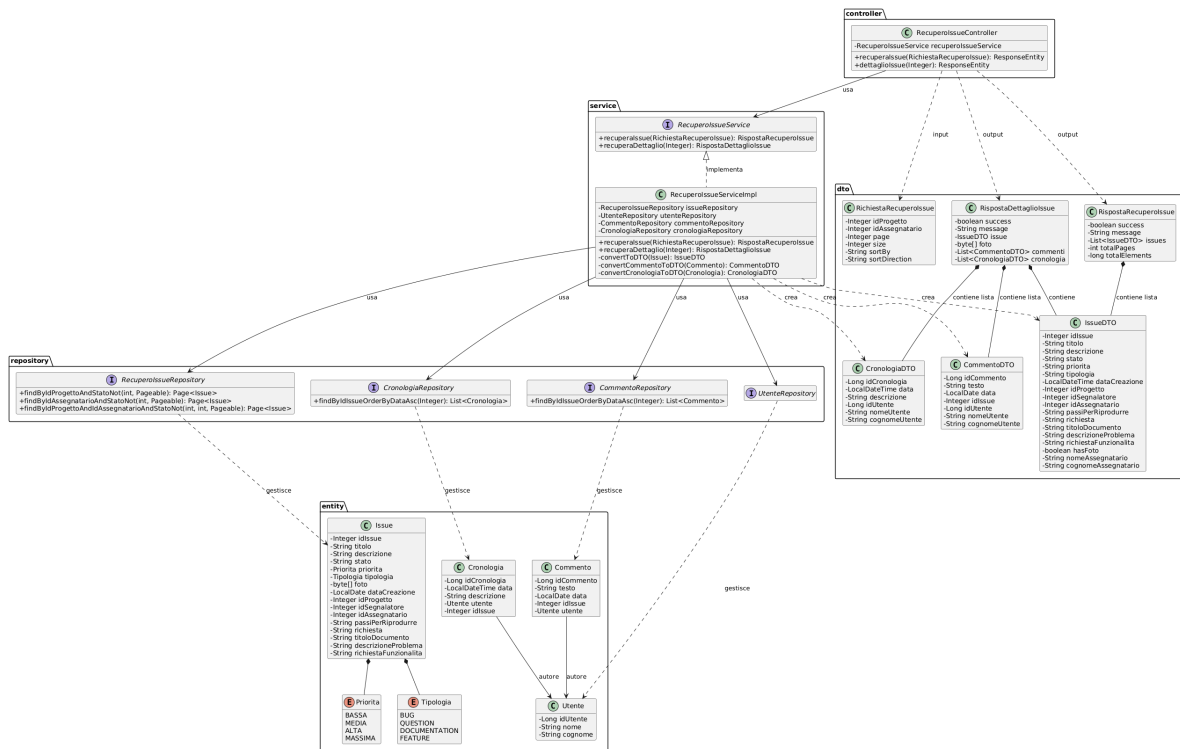


Figura 3.8: Diagramma UML servizio Recupero Issue



## 3.9 ModificaIssue-Service

Il modulo `ModificaIssue-Service` è il componente architetturale preposto alla gestione del *workflow* delle segnalazioni all'interno della piattaforma BugBoard. Oltre alla logica di business per il tracciamento delle modifiche, questo servizio si distingue per l'integrazione con componenti trasversali di sicurezza.

### 3.9.1 Architettura Modulare e Libreria Condivisa (JwtCommon)

Per garantire coerenza nelle politiche di sicurezza e rispettare il principio DRY (*Don't Repeat Yourself*), il servizio non implementa internamente la logica di validazione dei token, ma importa una dipendenza esterna personalizzata denominata `JwtCommon`.

Come visibile nel file di configurazione `pom.xml`, il modulo dichiara una dipendenza verso l'artefatto `it.unina.bugboard:jwt-common`. Questa scelta progettuale permette di:

- **Centralizzare la logica JWT:** La classe `JwtUtils`, proveniente da questa libreria, incapsula le operazioni critiche di parsing, verifica della firma e estrazione dei claim (ID utente, Nome, Cognome).
- **Decoppiare il controller:** Il `ModificaIssueController` delega completamente la "comprensione" del token alla libreria esterna, concentrandosi esclusivamente sull'orchestrazione del processo di modifica.

### 3.9.2 Logica di Business: Change Detection e Auditing

Il cuore del servizio risiede nella classe `ModificaIssueServiceImpl`, che implementa un meccanismo di *Change Detection* granulare. Il sistema esegue un confronto puntuale tra lo stato attuale salvato su database e i nuovi valori inviati tramite il DTO `RichiestaModificaIssue`. Solo le variazioni effettive di `stato` o `priorita` innescano una doppia operazione di scrittura: l'aggiornamento dell'entità `Issue` e l'inserimento di un record immutabile nella tabella `Cronologia`, garantendo un audit trail completo delle operazioni.



### 3.9.3 Diagramma UML ModificaIssueService

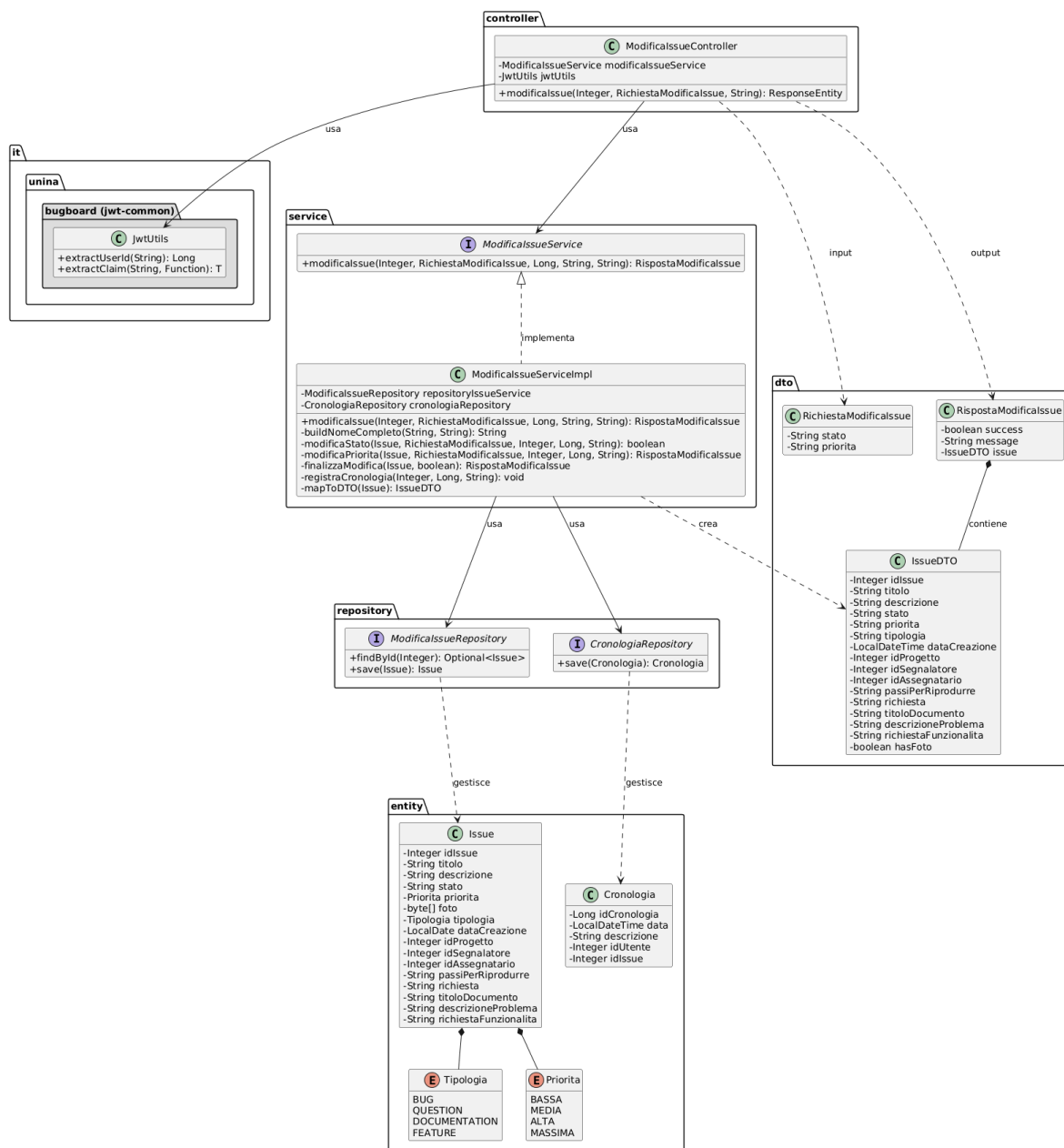


Figura 3.9: Diagramma UML servizio Modifica Issue



## 3.10 CommentaIssue-Service

Il modulo `CommentaIssue-Service` abilita la collaborazione tra gli utenti permettendo l'inserimento di commenti sulle segnalazioni. A differenza dei moduli precedenti, questo servizio evidenzia chiaramente l'adozione di pattern tipici delle architetture a microservizi distribuiti, ottimizzando le performance di scrittura e delegando aspetti di sicurezza all'infrastruttura.

### 3.10.1 Gestione dell'Identità: Un Approccio Ibrido

L'analisi del controller rivela un cambiamento strategico rispetto al modulo di Modifica. Invece di decodificare il token JWT internamente, il servizio adotta il pattern **Gateway Offloading**, aspettandosi l'identificativo utente tramite l'header `X-User-Id`.

Questa divergenza architetturale rispetto al servizio `ModificaIssue` non è accidentale, ma risponde al principio del *Context-Aware Security*:

- **ModificaIssue (Deep Inspection):** In quel contesto, il sistema doveva generare log di audit "parlanti" (es. *"Mario Rossi ha modificato..."*), rendendo necessaria la decodifica del token per estrarre Nome e Cognome dai *Claims*.
- **CommentaIssue (Gateway Offloading):** In questo contesto, il requisito è limitato alla creazione di un legame relazionale (Foreign Key) tra commento e autore. Non servendo dati anagrafici, il servizio evita il costo computazionale del parsing del token, fidandosi dell'ID validato e inoltrato dall'API Gateway.

### 3.10.2 Regole di Business e Validazione di Stato

Il service layer (`CommentaIssueServiceImpl`) non si limita a persistere il dato, ma applica regole di business stringenti per garantire la coerenza del processo:

- **Integrità Referenziale:** Verifica l'esistenza della Issue target.
- **Controllo di Stato (State Guard):** Implementa un controllo bloccante che impedisce l'aggiunta di commenti su segnalazioni nello stato `CLOSED`. Questo vincolo assicura che le discussioni vengano congelate al termine del ciclo di vita del bug.

### 3.10.3 Ottimizzazione delle Relazioni JPA (Proxy Pattern)

Un dettaglio implementativo di rilievo è l'uso di oggetti "proxy" per le relazioni. Poiché il servizio riceve solo l'ID numerico dell'utente (grazie all'approccio *Gateway Offloading*), per associare l'autore senza eseguire query superflue sulla tabella `Utenti`, viene istanziata un'entità parziale:

```
Utente utente = new Utente();
utente.setIdUtente(userId);
nuovoCommento.setUtente(utente);
```

JPA utilizza questo riferimento per impostare correttamente la chiave esterna nel database durante il salvataggio, evitando un round-trip di lettura (`SELECT`) e massimizzando il throughput di scrittura.



### 3.10.4 Diagramma UML CommentaIssueService

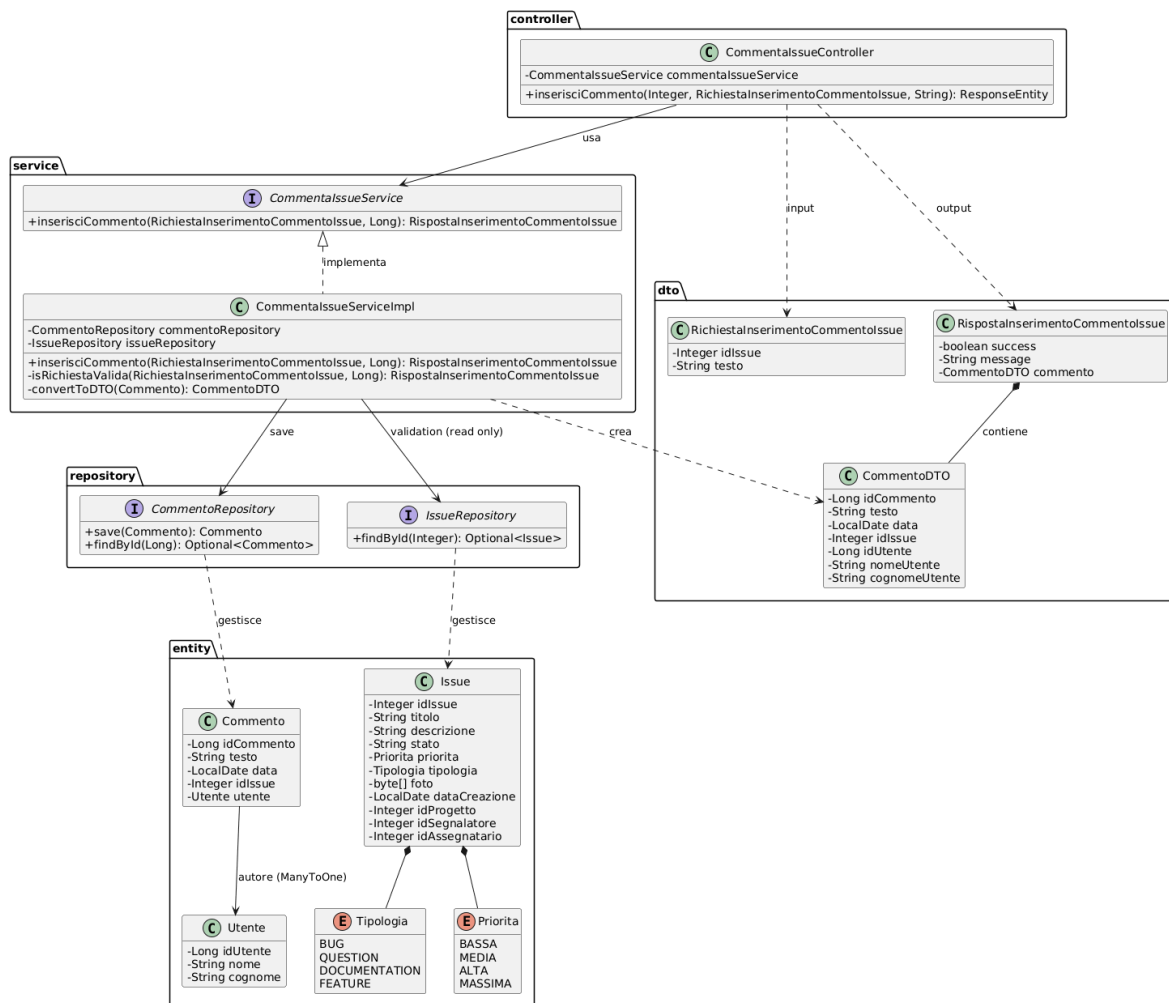


Figura 3.10: Diagramma UML servizio Commenta Issue



## 3.11 Libreria Condivisa: JWT-Common

Al fine di evitare la duplicazione del codice di sicurezza e garantire politiche di autenticazione uniformi su tutti i microservizi, il progetto adotta un approccio modulare estraendo la logica di gestione dei token in una libreria dedicata denominata `jwt-common`. Questo modulo agisce come un componente *Cross-Cutting Concern*, centralizzando le operazioni crittografiche e la gestione del ciclo di vita dei JSON Web Token (JWT).

### 3.11.1 Centralizzazione della Logica Crittografica

La classe principale, `JwtUtils`, incapsula l'intera complessità derivante dall'uso della libreria *JJWT* (Java JWT). Tra le caratteristiche implementative di rilievo si segnalano:

- **Gestione Robusta del Segreto:** Il costruttore della classe implementa una logica adattiva per la decodifica della chiave segreta (iniettata tramite `@Value("${jwt.secret}")`), supportando formati Base64URL, Base64 standard o UTF-8 raw. Questo garantisce flessibilità nella configurazione degli ambienti di deployment (es. Secret di Kubernetes o variabili d'ambiente Docker).
- **Custom Claims:** Oltre ai claim standard (Subject, Expiration), la libreria è progettata per includere nel payload del token informazioni di dominio specifiche del progetto BugBoard, quali `userId`, `nome`, `cognome` e `role`. Questa scelta architetturale permette ai servizi consumatori (come `ModificaIssue-service`) di ottenere il contesto dell'utente direttamente dal token senza dover interrogare il database utenti, migliorando le performance.

### 3.11.2 Gestione delle Eccezioni

La libreria definisce un'eccezione personalizzata `InvalidJwtException` (estensione di `RuntimeException`). Questo permette di disaccoppiare i servizi chiamanti dalle eccezioni specifiche della libreria sottostante (JJWT), fornendo un'interfaccia di errore unificata e chiara in caso di token scaduti, malformati o con firma non valida.

### 3.11.3 Integrazione

Essendo impacchettata come file JAR e gestita tramite Maven, questa libreria viene importata come dipendenza nei file `pom.xml` dei vari microservizi. Grazie all'annotazione `@Component` di Spring su `JwtUtils`, essa viene automaticamente rilevata dal Component Scan dei servizi ospitanti, rendendo le sue funzionalità immediatamente disponibili tramite Dependency Injection.





### 3.11.4 Diagramma UML JwtCommon

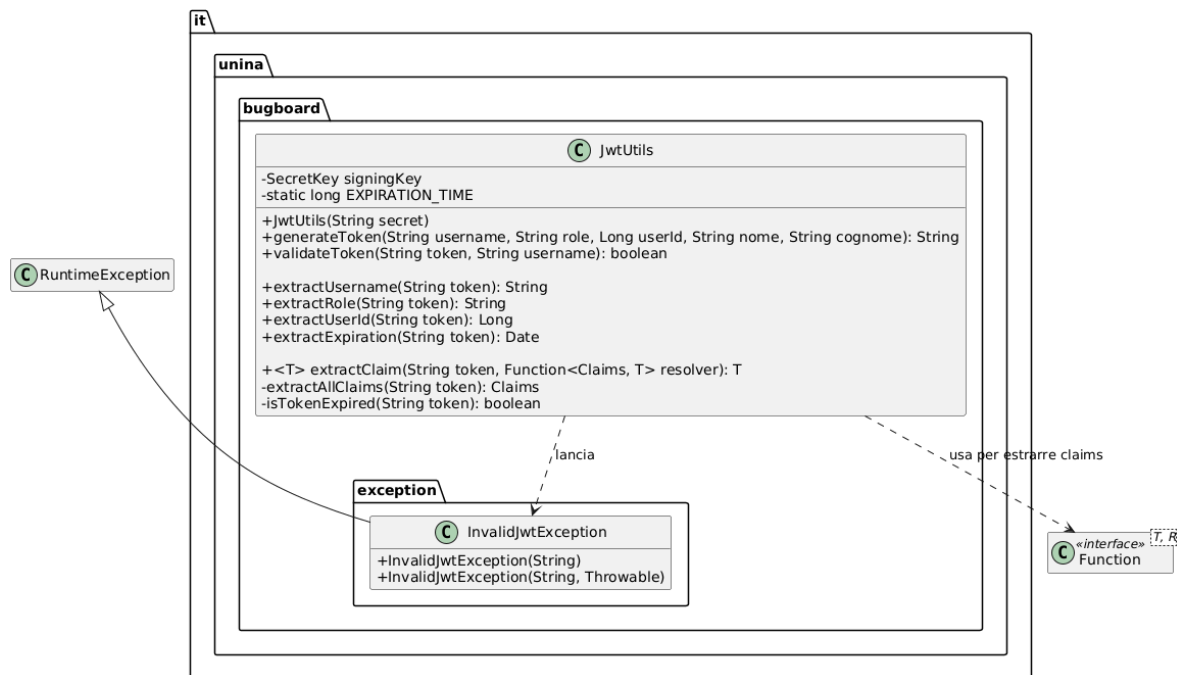


Figura 3.11: Diagramma UML libreria JwtCommon



## 3.12 Livello API Gateway

Abbiamo finora analizzato i primi due livelli dell'architettura proposta, ovvero il livello dedito alla persistenza dei dati e quello che implementa la logica di business. Ora passiamo al terzo livello dell'architettura, il livello **API Gateway**, che funge da tramite essenziale tra la logica applicativa distribuita e il livello di presentazione (Frontend).

In un ecosistema a microservizi, esporre direttamente gli endpoint di ciascun container comporterebbe notevoli criticità di sicurezza e complessità gestionale. L'API Gateway risolve queste problematiche agendo come *Single Entry Point* per tutto il traffico in entrata, implementando tre funzioni architetturali critiche:

1. **Reverse Proxy e Routing:** Maschera la topologia della rete interna, disaccoppiando il client dagli indirizzi fisici dei servizi.
2. **Gateway Offloading:** Centralizza le responsabilità trasversali (Cross-Cutting Concerns) come l'autenticazione e il CORS, alleggerendo i singoli microservizi.
3. **Protocol Translation:** Offre al client un'interfaccia API omogenea (REST) indipendente dalla frammentazione del backend.

### 3.12.1 Implementazione con Spring Cloud Gateway

Per realizzare questo componente è stata scelta la tecnologia **Spring Cloud Gateway**, una soluzione non bloccante basata su Project Reactor. Il cuore della sicurezza risiede nella classe `JwtAuthenticationFilter`, un filtro globale che intercetta ogni richiesta in ingresso applicando una pipeline di validazione rigorosa:

- **Deep Token Inspection:** Il filtro estrae il Bearer Token dall'header `Authorization` e ne verifica la firma crittografica utilizzando la libreria condivisa `JwtUtils`.
- **Role-Based Access Control (RBAC):** Per endpoint sensibili, come la registrazione utenti, il gateway verifica preventivamente che il token contenga il ruolo `admin`, bloccando richieste non autorizzate prima che raggiungano il backend.
- **Header Injection:** Una volta validata l'identità, il filtro arricchisce la richiesta iniettando header custom (`X-User-Id`, `X-User-Name`). Questo meccanismo concretizza il pattern di *Offloading*, permettendo ai servizi a valle (come `CommentaIssue`) di consumare l'identità utente senza dover ridecodificare il token.

### 3.12.2 Deployment e Configurazione Dinamica su Azure

L'architettura del Gateway è stata progettata per essere *Cloud Native* e agnostica rispetto all'ambiente. Seguendo la metodologia *12-Factor App*, la configurazione delle rotte non è statica all'interno dell'artefatto (JAR), ma viene iniettata dinamicamente a runtime tramite il file di orchestrazione `docker-compose.yml`.

Questa strategia di **Externalized Configuration** offre la massima flessibilità. Le rotte sono definite attraverso variabili d'ambiente che sfruttano il *Relaxed Binding* di Spring Boot per sovrascrivere le impostazioni di default. Di seguito, un estratto della configurazione di produzione utilizzata sulla Virtual Machine Azure:

environment:

```
# Rotta per il Login Service
SPRING_CLOUD_GATEWAY_ROUTES_0_ID: login-service
SPRING_CLOUD_GATEWAY_ROUTES_0_URI: http://login-service:${SERVER_PORT_LOGIN}
SPRING_CLOUD_GATEWAY_ROUTES_0_PREDICATES_0: Path=/api/auth/**
```



```
# Rotta per Inserimento Issue (Routing per Metodo)
SPRING_CLOUD_GATEWAY_ROUTES_2_ID: inserimento-issue-service
SPRING_CLOUD_GATEWAY_ROUTES_2_URI: http://inserimento-issue-service:...
SPRING_CLOUD_GATEWAY_ROUTES_2_PREDICATES_0: Path=/api/issues
SPRING_CLOUD_GATEWAY_ROUTES_2_PREDICATES_1: Method=POST
```

L'analisi di questa configurazione evidenzia due pattern avanzati:

1. **Service Discovery Statico:** Gli URI di destinazione utilizzano i nomi dei servizi Docker (es. `http://login-service`), che vengono risolti automaticamente dal DNS interno della rete `bugboard-network`, garantendo l'isolamento dei container.
2. **Method-Based Routing:** Il Gateway discrimina il traffico non solo in base al percorso URL ma anche in base al verbo HTTP. Come mostrato nell'esempio sopra, le richieste `POST` su `/api/issues` vengono instradate al servizio di scrittura (`inserimento-issue-service`), mentre le richieste `GET` sullo stesso path (configurate in altre variabili) vengono dirette al servizio di lettura (`recupero-issue-service`).

In conclusione, questa configurazione permette di modificare la topologia dei servizi e le politiche di instradamento direttamente dall'infrastruttura di deployment, senza necessità di ricompilare il codice sorgente del Gateway.

### 3.13 Il Livello di Presentazione: Frontend Desktop

A completamento dell'architettura distribuita descritta nei capitoli precedenti, è stato sviluppato un livello di presentazione (Frontend) incaricato di fornire all'utente finale un'interfaccia grafica interattiva e intuitiva per la gestione del ciclo di vita dei bug.

La scelta tecnologica è ricaduta su **JavaFX**, un framework moderno per lo sviluppo di rich client application, che ha permesso di realizzare un'applicazione desktop modulare, reattiva e fortemente tipizzata.

#### 3.13.1 Architettura Model-View-Controller (MVC)

L'applicazione frontend adotta rigorosamente il pattern architetturale **MVC (Model-View-Controller)** per garantire una netta separazione delle responsabilità:

- **View (FXML e CSS):** L'interfaccia utente è definita in modo dichiarativo tramite file `.fxml`. Ogni schermata (Login, Home, Dettaglio Issue) è un modulo isolato che descrive la struttura gerarchica dei componenti grafici. Lo stile visivo è delegato a fogli di stile CSS separati, garantendo un *look-and-feel* coerente e professionale.
- **Controller:** Per ogni vista FXML esiste un corrispondente Java Controller (es. `LoginController`, `AllIssuesController`). Questi componenti gestiscono l'interazione con l'utente, la validazione degli input e la logica di navigazione, agendo da collante tra l'interfaccia e i dati.
- **Model e Service Layer:** I dati sono modellati attraverso classi DTO (*Data Transfer Objects*) speculari a quelle del backend (es. `IssueDTO`, `CommentoDTO`). La logica di recupero e persistenza è incapsulata in classi "ApiService" (es. `LoginApiService`, `IssueApiService`), che astraggono la complessità delle chiamate HTTP verso l'API Gateway.

#### 3.13.2 Comunicazione e Gestione dello Stato

Sebbene il frontend agisca come un *Thin Client*, delegando la logica di business ai microservizi, esso gestisce in autonomia aspetti critici della sicurezza e della navigazione utente.



## Gestione della Sessione (JWT)

Essendo il backend *stateless*, il client ha la responsabilità di mantenere lo stato dell'autenticazione. È stata implementata una classe Singleton **SessionManager** che agisce come "cassaforte in memoria" per il Token JWT ricevuto al momento del login. Questo componente rende il token accessibile trasversalmente a tutti i service, i quali provvedono a iniettarlo automaticamente nell'header **Authorization: Bearer** di ogni richiesta HTTP successiva.

## Routing Dinamico

La navigazione tra le diverse schermate non avviene tramite istanziazione diretta, ma è orchestrata da un componente centrale denominato **SceneRouter**. Questa classe gestisce il caricamento efficiente delle risorse FXML e il passaggio di parametri tra le scene (ad esempio, trasferendo l'ID di una issue dalla lista alla schermata di dettaglio), centralizzando inoltre la gestione delle eccezioni di caricamento.

### 3.13.3 Mockup ad Alta Fedeltà e Flusso Utente

Di seguito vengono presentati i mockup ad alta fedeltà dell'applicazione finale. Le schermate illustrano l'intero flusso operativo, dalla fase di autenticazione alla gestione dettagliata delle segnalazioni.

Per massimizzare la leggibilità del diagramma riportato in Figura 3.12, sono state omesse alcune linee di collegamento ridondanti. Si specifica tuttavia che l'interfaccia mantiene una coerenza funzionale in ogni contesto:

- **Azioni Globali:** Comandi come "Inserisci Issue" evocano la medesima procedura modale indipendentemente dalla schermata di origine (sia essa la HomePage o il Dettaglio).
- **Navigazione Storica:** Il pulsante "Indietro" interagisce con lo stack di navigazione dello **SceneRouter** per garantire sempre il ritorno allo stato immediatamente precedente.

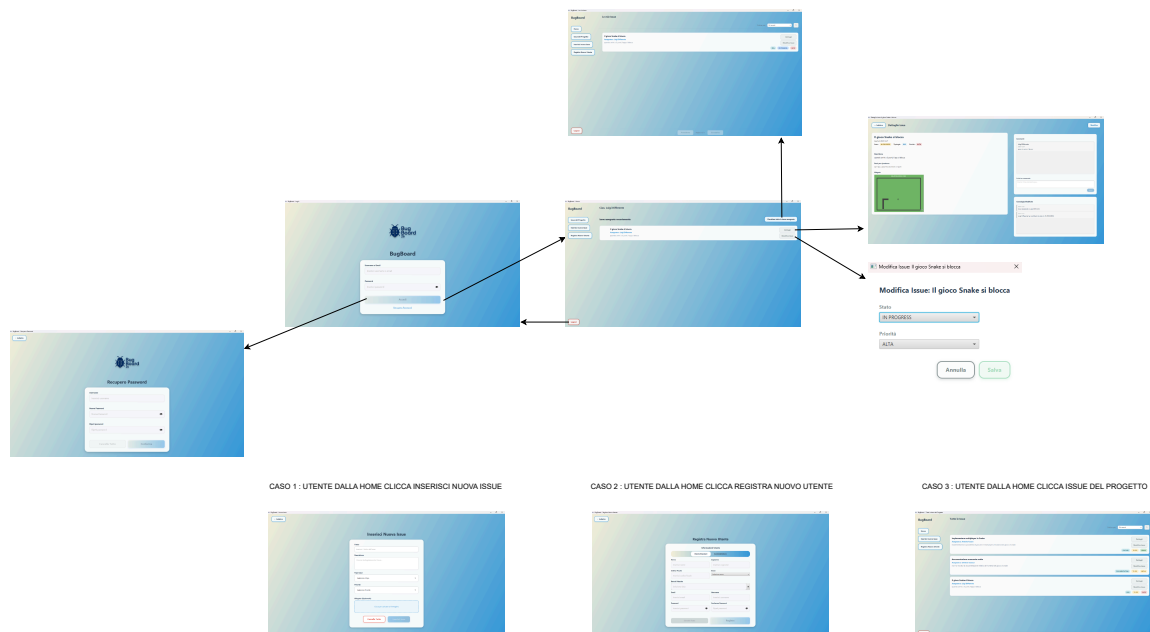


Figura 3.12: Mockup ad Alta Fedeltà.  
([Clicca qui per visualizzare l'immagine](#))



## 3.14 Design dell'Esperienza Utente (UX) e Usabilità

Oltre alla struttura architetturale, grande attenzione è stata posta nel design dell'interazione (*Interaction Design*). L'obiettivo è stato ridurre il carico cognitivo dell'utente e prevenire errori di input prima ancora che i dati vengano inviati al server. Dall'analisi dei Controller implementati emergono quattro pilastri fondamentali dell'usabilità del sistema:

### 3.14.1 Validazione Reattiva e Feedback Visivo

Il sistema abbandona il paradigma della validazione "post-submit" in favore di un approccio reattivo basato sulle *JavaFX Properties*. Attraverso l'uso di `BooleanBinding`, lo stato di abilitazione dei pulsanti di azione (Login, Registra, Inserisci) è legato indissolubilmente alla validità dei campi di input.

Ad esempio, nella schermata di inserimento utente, il pulsante "Registra" rimane disabilitato fin tanto che tutti i campi obbligatori non sono compilati e le password non coincidono. Questo pattern, noto come *Preventing User Error*, fornisce un feedback immediato: se il pulsante è cliccabile, i dati sono formalmente corretti.

Inoltre, per migliorare la percezione degli errori, sono state implementate micro-interazioni visive: in caso di login fallito, il form esegue una transizione di "shake" (scuotimento) laterale accompagnata da messaggi di errore inline, simulando il comportamento naturale di diniego.

### 3.14.2 Efficienza e Accessibilità da Tastiera

Per supportare un utilizzo intensivo da parte di utenti esperti (Power Users), l'applicazione è interamente navigabile senza l'ausilio del mouse. È stata implementata una logica di *Focus Traversal* personalizzata:

- **Navigazione Sequenziale:** La pressione del tasto `ENTER` sposta automaticamente il focus al campo logico successivo (es. da Nome a Cognome, poi a Codice Fiscale), fluidificando il data entry.
- **Scorciatoie Avanzate:** Nella schermata di inserimento Issue, dove è presente un'area di testo multilinea per la descrizione, è stato introdotto il key-combination `CTRL + ENTER` per inviare direttamente il form, distinguendo l'azione di invio da quella di "a capo".

Inoltre sono stati implementati altri shortcut mnemonici per le operazioni più comuni.

### 3.14.3 Interfacce Dinamiche (Progressive Disclosure)

Per gestire la complessità dei dati senza sovraccaricare l'interfaccia, è stato adottato il pattern della *Progressive Disclosure*. Nel controller `InsertIssueController`, il form si adatta dinamicamente in base alla "Tipologia" selezionata dall'utente. Se si seleziona `BUG`, appare il campo "Passi per riprodurre"; se si seleziona `DOCUMENTATION`, appaiono i campi relativi al documento. Questa logica polimorfica lato frontend assicura che l'utente veda solo i campi pertinenti al contesto attuale, mantenendo l'interfaccia pulita.

### 3.14.4 Gestione dello Stato e Integrità

Infine, un dettaglio di rilievo riguarda la gestione delle modifiche. Nel popup di modifica stato/priorità, il pulsante "Salva" si attiva esclusivamente se i valori selezionati differiscono da quelli originali dell'Issue. Questo controllo (*Dirty Checking*) previene chiamate API inutili e comunicazioni ridondanti verso il backend, ottimizzando l'uso della rete.



## Capitolo 4

# Processo di Sviluppo e Qualità del Software

Procediamo con l'indicare l'utilizzo di strumenti di versioning e qualità del software.

### 4.1 Strumenti di versioning

Nel corso dello sviluppo del progetto sono stati utilizzati strumenti di versioning come Git, per gestire il codice sorgente e collaborare con i membri del team. La repository pubblicata è facilmente accessibile su GitHub tramite [questo link](#).

#### 4.1.1 Code Frequency

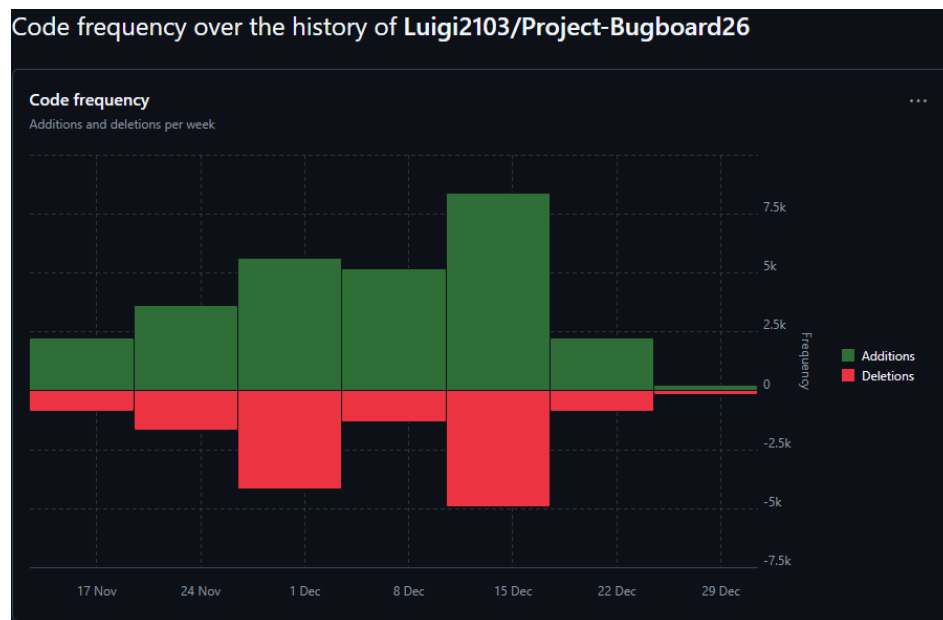


Figura 4.1: Code Frequency



### 4.1.2 Statistiche commit

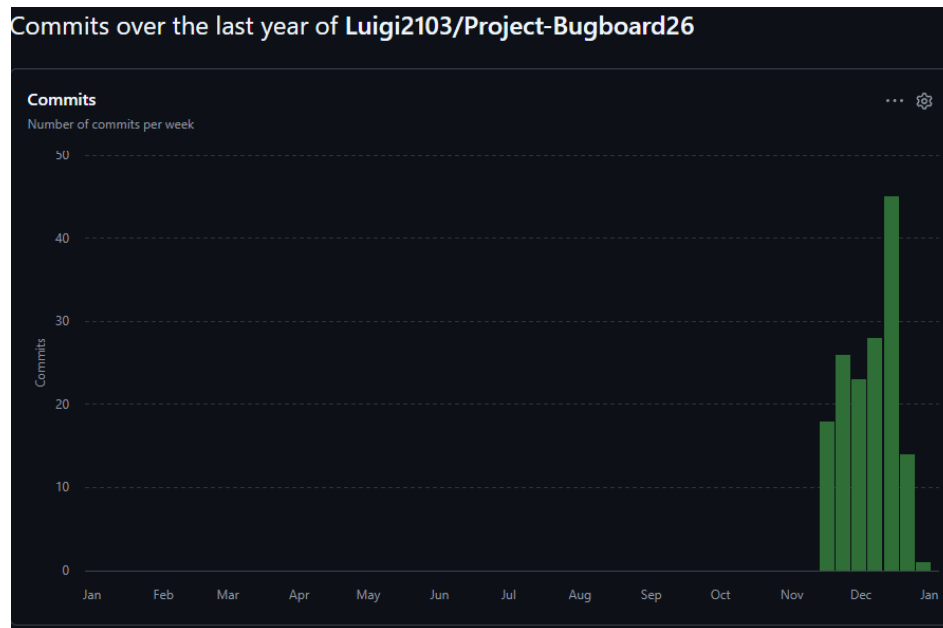


Figura 4.2: Statistiche commit



### 4.1.3 Statistiche commit per collaboratore

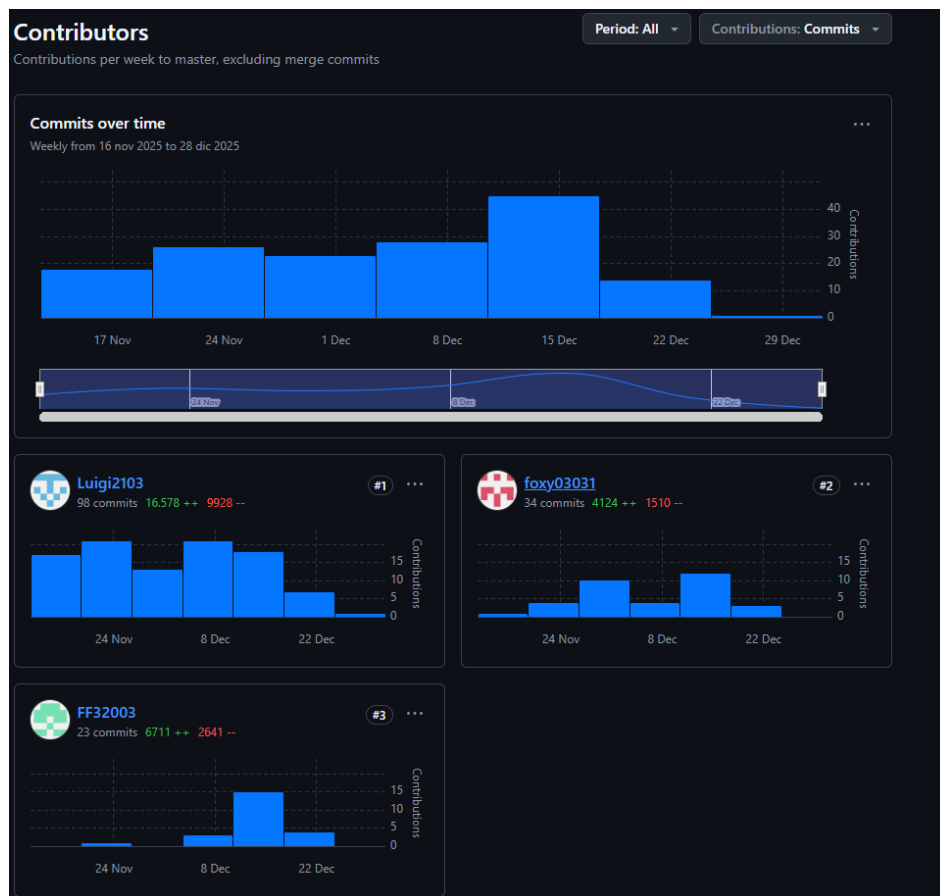


Figura 4.3: Statistiche commit per collaboratore





## 4.2 Strumenti di Qualità e Scelte Architettureali

Per garantire elevati standard di manutenibilità e sicurezza del software, il progetto ha affiancato all'analisi automatizzata l'adozione di rigorosi pattern architetturali, configurando gli strumenti in modo coerente con il paradigma a microservizi.

### 4.2.1 Analisi Statica con SonarQube e Docker

L'ispezione della qualità del codice backend è stata affidata a **SonarQube**, distribuito tramite container Docker per garantire un ambiente di analisi isolato e riproducibile. L'analisi ha coperto metriche fondamentali quali il rilevamento di bug, vulnerabilità di sicurezza e "Code Smells", permettendo di mantenere il debito tecnico sotto controllo.

### 4.2.2 Gestione dei Falsi Positivi: Il caso dei DTO

In fase di configurazione dei Quality Gate, si è reso necessario escludere i package `*.dto` e `*.entity` dalle metriche di **Duplicazione del Codice** (CPD - Copy Paste Detector).

L'uso della libreria **Lombok** ha già eliminato il codice boilerplate (getter, setter, costruttori), riducendo drasticamente la verbosità delle classi. Tuttavia, SonarQube identificava erroneamente come "duplicazione nociva" la presenza di classi speculari (come `IssueDTO`) in microservizi diversi (es. `InserimentoIssue-service` e `ModificaIssue-service`).

Questa ridondanza è in realtà una scelta architetturale precisa, nota come pattern **Bounded Context**:

- **Disaccoppiamento (Decoupling):** Se avessimo creato una libreria condivisa (*Shared Kernel*) contenente tutti i DTO, ogni modifica a un oggetto per le esigenze di un singolo servizio avrebbe costretto al ricompilamento e rilascio di tutti gli altri servizi.
- **Indipendenza del Dominio:** Accettare la duplicazione delle classi DTO permette a ogni microservizio di evolvere la propria interfaccia dati in autonomia, senza introdurre dipendenze rigide verso gli altri componenti del sistema.

Pertanto, l'esclusione di queste classi dall'analisi della duplicazione non è un'omissione, ma una configurazione necessaria per supportare l'indipendenza dei microservizi.

Di seguito viene indicato il risultato dell'analisi statica:

### 4.2.3 Risultati analisi

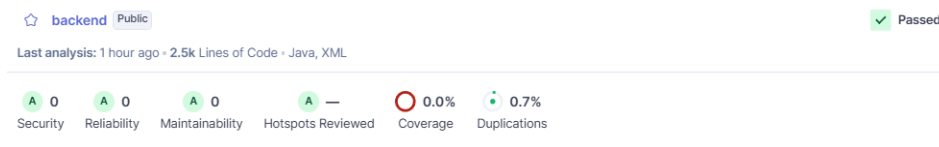


Figura 4.4: Risultati analisi

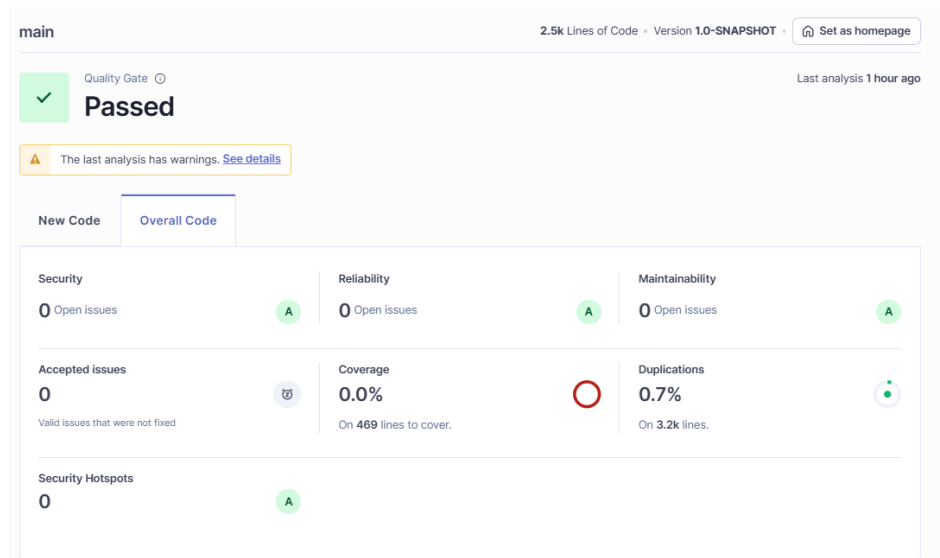


Figura 4.5: Risultati analisi

Per una corretta interpretazione dei risultati mostrati nelle dashboard di SonarQube, è opportuno chiarire alcuni aspetti tecnici relativi all'attuale fase di sviluppo e all'ambiente di esecuzione.

#### 4.2.4 Copertura dei Test (Code Coverage)

Come da pianificazione del ciclo di vita del progetto, l'attuale release si focalizza sul consolidamento architetturale e sull'implementazione delle funzionalità core. La fase di *Unit Testing* e *Integration Testing* è programmata per lo sprint successivo. Di conseguenza, le metriche relative alla **Code Coverage** (attualmente pari allo 0%) non sono indicative della qualità del codice in questa fase e sono state temporaneamente escluse dai criteri di fallimento del Quality Gate.

#### 4.2.5 Warning "Missing Blame Information"

I log di analisi riportano un warning ricorrente: *"Missing blame information"*. Questo messaggio è un effetto collaterale atteso dell'esecuzione di SonarQube in ambiente containerizzato. Per ottimizzare i tempi di build e ridurre la dimensione delle immagini Docker, la cartella di metadata `.git` non viene copiata nel contesto di analisi. Ciò impedisce allo scanner di accedere allo storico dei commit (autore e data di modifica per riga), ma non pregiudica in alcun modo l'efficacia dell'analisi statica per il rilevamento di bug, vulnerabilità e code smells.



## Capitolo 5

# Fase di Testing

In questo capitolo descriviamo le strategie di **Verifica e Validazione (V&V)** adottate per garantire la qualità complessiva del software. Il processo di testing è stato articolato su due livelli complementari:

- **Testing Funzionale e Design dei Test (Black Box):**

Per la definizione dei casi di test (*Test Cases*), abbiamo adottato un approccio *Black Box*, basandoci sulle specifiche funzionali. In particolare:

- Abbiamo redatto un **Test Plan** formale per la funzionalità di *Modifica Issue*.
- Abbiamo applicato la tecnica del **Partizionamento in Classi di Equivalenza** (R-WECT) per coprire scenari di successo e di errore.
- **GUI Testing Automatizzato:** Abbiamo utilizzato il framework **TestFX** per automatizzare le interazioni con l'interfaccia grafica JavaFX, verificando il comportamento dinamico dei controller (es. abilitazione pulsanti, messaggi di errore) in isolamento dal backend.
- Abbiamo condotto sessioni di **Usability Testing** con utenti reali.

- **Testing Strutturale e di Unità (White Box):**

Abbiamo implementato **Test di Unità Automatici** con **JUnit 5** e **Mockito** per verificare la correttezza logica dei metodi "non banali", massimizzando la **Branch Coverage**.

### 5.1 Analisi delle Classi di Equivalenza

Nella fase di analisi del dominio per il metodo `modificaIssue`, abbiamo adottato un approccio eterogeneo per la gestione dei tipi di dato, al fine di sperimentare due diverse strategie di design per la persistenza:

- **Strategia "Strong Typing" (Campo Priorità):** Utilizzo di *Enum* sia in Java che nel Database per garantire la sicurezza dei tipi (*Compile-time Safety*).
- **Strategia "Soft Validation" (Campo Stato):** Mappatura a *Stringa* in Java verso un *ENUM* nel Database. Questa scelta privilegia il disaccoppiamento e permette una validazione personalizzata (*Custom Validation*) nel Service.

Questa differenziazione ci ha permesso di definire le seguenti classi di equivalenza:

### 5.2 Test Plan: Modifica Issue

Riportiamo il piano di test funzionale derivato dall'analisi delle classi di equivalenza. Ogni caso di test corrisponde a un metodo implementato nella suite JUnit.



| Parametro              | Classi Valide (Valid)                                                          | Classi Non Valide (Invalid)                                                                         |
|------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>ID Issue</b>        | <i>C1</i> : ID presente nel Database (es. 1)                                   | <i>C2</i> : ID non esistente (es. 999)<br><i>C3</i> : ID nullo o negativo                           |
| <b>Stato (String)</b>  | <i>C4</i> : Stringa corrispondente a uno stato lecito ("TO-DO", "IN PROGRESS") | <i>C5</i> : Stringa vuota ("")<br><i>C6</i> : Stringa arbitraria non riconosciuta (es. "ANNULLATO") |
| <b>Priorità (Enum)</b> | <i>C7</i> : Valori dell'Enum (BASSA, MEDIA, ALTA)                              | <i>C8</i> : Valori non convertibili<br><i>C9</i> : Null                                             |

Tabella 5.1: Analisi Comparata delle Classi di Equivalenza

| ID           | Descrizione         | Input                                                         | Oracolo (Atteso)                                                                             |
|--------------|---------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>TC-01</b> | Cambio Stato        | <i>Stato</i> : OPEN<br><i>Richiesta</i> : Stato="IN_PROGRESS" | <b>Esito</b> : Successo<br><b>Stato Finale</b> : IN_PROGRESS                                 |
| <b>TC-02</b> | Cambio Priorità     | <i>Priorità</i> : BASSA<br><i>Richiesta</i> : Priorità="ALTA" | <b>Esito</b> : Successo<br><b>Priorità Finale</b> : ALTA                                     |
| <b>TC-03</b> | Cambio Multiplo     | <i>Richiesta</i> : Stato="IN_PROGRESS", Priorità="ALTA"       | <b>Esito</b> : Successo su entrambi i campi                                                  |
| <b>TC-04</b> | Priorità Non Valida | <i>Richiesta</i> : Priorità="SUPER_URGENTE"                   | <b>Esito</b> : Fallimento<br><b>Msg</b> : "Priorità non valida"                              |
| <b>TC-05</b> | Input Vuoto         | <i>Richiesta</i> : Priorità=""                                | <b>Esito</b> : Fallimento<br><b>Msg</b> : "Priorità non valida"                              |
| <b>TC-06</b> | Nessuna Modifica    | <i>Stato</i> : OPEN<br><i>Richiesta</i> : Stato="OPEN"        | <b>Esito</b> : Successo<br><b>Msg</b> : "Nessuna modifica"<br><b>DB</b> : Nessun salvataggio |
| <b>TC-07</b> | Issue Non Trovata   | <i>ID Issue</i> : 999                                         | <b>Esito</b> : Fallimento<br><b>Msg</b> : "Issue non trovata"                                |

Tabella 5.2: Test Plan per la funzionalità di Modifica Issue

## 5.3 Test di Unità

Di seguito riportiamo l'implementazione completa dei test di unità sviluppati utilizzando **JUnit 5** e **Mockito**.

### 5.3.1 Test 1: Modifica Issue

Questo test copre la logica di aggiornamento condizionale.

```

1 package it.unina.bugboard.service;
2
3 import it.unina.bugboard.dto.RichiestaModificaIssue;
4 import it.unina.bugboard.dto.RispostaModificaIssue;
5 import it.unina.bugboard.entity.Issue;
6 import it.unina.bugboard.entity.Priorita;
7 import it.unina.bugboard.repository.CronologiaRepository;

```



```
8 import it.unina.bugboard.repository.ModificaIssueRepository;
9 import org.junit.jupiter.api.Test;
10 import org.junit.jupiter.api.extension.ExtendWith;
11 import org.mockito.InjectMocks;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14
15 import java.util.Optional;
16
17 import static org.junit.jupiter.api.Assertions.*;
18 import static org.mockito.ArgumentMatchers.any;
19 import static org.mockito.Mockito.*;
20
21 @ExtendWith(MockitoExtension.class)
22 class ModificaIssueServiceImplTest {
23
24     @Mock
25     private ModificaIssueRepository repository;
26
27     @Mock
28     private CronologiaRepository cronologiaRepository;
29
30     @InjectMocks
31     private ModificaIssueServiceImpl service;
32
33     @Test
34     void testModificaIssue_CambioStato_Successo() {
35         // ARRANGE
36         Integer issueId = 1;
37         Long userId = 100L;
38
39         Issue issue = new Issue();
40         issue.setIdIssue(issueId);
41         issue.setStato("OPEN");
42
43         when(repository.findById(issueId)).thenReturn(Optional.of(issue));
44         when(repository.save(any(Issue.class))).thenAnswer(i -> i.getArguments()[0]);
45
46         RichiestaModificaIssue richiesta = new RichiestaModificaIssue();
47         richiesta.setStato("IN_PROGRESS");
48
49         // ACT
50         RispostaModificaIssue response = service.modificaIssue(issueId, richiesta,
51             userId, "Mario", "Rossi");
52
53         // ASSERT
54         assertTrue(response.isSuccess());
55         assertEquals("IN_PROGRESS", response.getIssue().getStato());
56         verify(repository, times(1)).save(any(Issue.class));
57         verify(cronologiaRepository, times(1)).save(any());
58     }
59
60     @Test
61     void testModificaIssue_CambioStatoVuoto_Fallimento() {
62         // ARRANGE
63         Integer issueId = 1;
64         Long userId = 100L;
65
66         Issue issue = new Issue();
67         issue.setIdIssue(issueId);
68         issue.setStato("OPEN");
69
70         when(repository.findById(issueId)).thenReturn(Optional.of(issue));
71
72         RichiestaModificaIssue richiesta = new RichiestaModificaIssue();
```



```
72     richiesta.setStato("");
73
74     // ACT
75     RispostaModificaIssue response = service.modificaIssue(issueId, richiesta,
76         userId, "Mario", "Rossi");
77
78     // ASSERT
79
80     assertFalse(response.isSuccess());
81     assertTrue(response.getMessage().contains("Stato non valido"));
82     verify(repository, never()).save(any(Issue.class));
83     verify(cronologiaRepository, never()).save(any());
84 }
85
86 @Test
87 void testModificaIssue_IssueNonTrovata_Fallimento() {
88     Integer idInesistente = 999;
89     when(repository.findById(idInesistente)).thenReturn(Optional.empty());
90
91     RichiestaModificaIssue richiesta = new RichiestaModificaIssue();
92     richiesta.setStato("IN_PROGRESS");
93
94     RispostaModificaIssue response = service.modificaIssue(idInesistente,
95         richiesta, 100L, "Mario", "Rossi");
96
97     assertFalse(response.isSuccess());
98     assertTrue(response.getMessage().toLowerCase().contains("non trovata"));
99     verify(repository, never()).save(any());
100 }
101
102 @Test
103 void testModificaIssue_CambioPriorita_Successo() {
104     Integer issueId = 1;
105     Issue issue = new Issue();
106     issue.setIdIssue(issueId);
107     issue.setPriorita(Priorita.BASSA);
108
109     when(repository.findById(issueId)).thenReturn(Optional.of(issue));
110     when(repository.save(any(Issue.class))).thenAnswer(i -> i.getArguments()[0]);
111
112     RichiestaModificaIssue richiesta = new RichiestaModificaIssue();
113     richiesta.setPriorita("ALTA");
114
115     RispostaModificaIssue response = service.modificaIssue(issueId, richiesta, 100
116         L, "Mario", "Rossi");
117
118     assertTrue(response.isSuccess());
119     assertEquals(Priorita.ALTA.name(), response.getIssue().getPriorita());
120     verify(cronologiaRepository, times(1)).save(any());
121 }
122
123 @Test
124 void testModificaIssue_PrioritaInvalida_Fallimento() {
125     Integer issueId = 1;
126     Issue issue = new Issue();
127     issue.setIdIssue(issueId);
128     when(repository.findById(issueId)).thenReturn(Optional.of(issue));
129
130     RichiestaModificaIssue richiesta = new RichiestaModificaIssue();
131     richiesta.setPriorita("SUPER_URGENTE");
132
133     RispostaModificaIssue response = service.modificaIssue(issueId, richiesta, 100
134         L, "Mario", "Rossi");
135
136     assertFalse(response.isSuccess());
```



```
133         assertTrue(response.getMessage().contains("Priorità non valida"));
134         verify(repository, never()).save(any());
135     }
136
137     @Test
138     void testModificaIssue_PrioritaNulla_Fallimento() {
139         Integer issueId = 1;
140         Issue issue = new Issue();
141         issue.setIdIssue(issueId);
142         when(repository.findById(issueId)).thenReturn(Optional.of(issue));
143
144         RichiestaModificaIssue richiesta = new RichiestaModificaIssue();
145         richiesta.setPriorita("");
146
147         RispostaModificaIssue response = service.modificaIssue(issueId, richiesta, 100
148             L, "Mario", "Rossi");
149
150         assertFalse(response.isSuccess());
151         assertTrue(response.getMessage().contains("Priorità non valida"));
152         verify(repository, never()).save(any());
153     }
154
155     @Test
156     void testModificaIssue_CambioStatoEPriorita_Successo() {
157         // ARRANGE
158         Integer issueId = 1;
159         Issue issue = new Issue();
160         issue.setIdIssue(issueId);
161         issue.setStato("OPEN");
162         issue.setPriorita(Priorita.BASSA);
163
164         when(repository.findById(issueId)).thenReturn(Optional.of(issue));
165         when(repository.save(any(Issue.class))).thenAnswer(i -> i.getArguments()[0]);
166
167         RichiestaModificaIssue richiesta = new RichiestaModificaIssue();
168         richiesta.setStato("IN_PROGRESS");
169         richiesta.setPriorita("ALTA");
170
171         // ACT
172         RispostaModificaIssue response = service.modificaIssue(issueId, richiesta, 100
173             L, "Mario", "Rossi");
174
175         // ASSERT
176         assertTrue(response.isSuccess());
177         assertEquals("IN_PROGRESS", response.getIssue().getStato());
178         assertEquals(Priorita.ALTA.name(), response.getIssue().getPriorita());
179         verify(repository, times(1)).save(any(Issue.class));
180         verify(cronologiaRepository, times(2)).save(any());
181     }
182
183     @Test
184     void testModificaIssue_NessunaModifica() {
185         Integer issueId = 1;
186         Issue issue = new Issue();
187         issue.setIdIssue(issueId);
188         issue.setStato("OPEN");
189         issue.setPriorita(Priorita.MEDIA);
190
191         when(repository.findById(issueId)).thenReturn(Optional.of(issue));
192
193         RichiestaModificaIssue richiesta = new RichiestaModificaIssue();
194         richiesta.setStato("OPEN");
195         richiesta.setPriorita("MEDIA");
```



```

196         RispostaModificaIssue response = service.modificaIssue(issueId, richiesta, 100
197             L, "Mario", "Rossi");
198
199         assertTrue(response.isSuccess());
200         assertTrue(response.getMessage().contains("Nessuna modifica"));
201         verify(repository, never()).save(any());
202     }
203 }

```

Listing 5.1: Test Completo per Modifica Issue

### 5.3.2 Test 2: Inserimento Commento

Questo test ha lo scopo di verificare il corretto funzionamento della fase di commento, assicurando che il messaggio inserito non sia vuoto e che la possibilità di commentare sia consentita esclusivamente per le issue non chiuse.

```

1 package it.unina.bugboard.service;
2
3 import it.unina.bugboard.dto.RichiestaInserimentoCommentoIssue;
4 import it.unina.bugboard.dto.RispostaInserimentoCommentoIssue;
5 import it.unina.bugboard.entity.Issue;
6 import it.unina.bugboard.repository.CommentoRepository;
7 import it.unina.bugboard.repository.IssueRepository;
8 import org.junit.jupiter.api.Test;
9 import org.junit.jupiter.api.extension.ExtendWith;
10 import org.mockito.InjectMocks;
11 import org.mockito.Mock;
12 import org.mockito.junit.jupiter.MockitoExtension;
13
14 import java.util.Optional;
15
16 import static org.junit.jupiter.api.Assertions.*;
17 import static org.mockito.ArgumentMatchers.any;
18 import static org.mockito.Mockito.*;
19
20 @ExtendWith(MockitoExtension.class)
21 class CommentaIssueServiceImplTest {
22
23     @Mock
24     private IssueRepository issueRepository;
25
26     @Mock
27     private CommentoRepository commentoRepository;
28
29     @InjectMocks
30     private CommentaIssueServiceImpl service;
31
32     @Test
33     void testInserisciCommento_SuIssueAperta_Successo() {
34         // ARRANGE
35         Integer issueId = 10;
36         Long userId = 50L;
37
38         // Simuliamo una Issue in stato "OPEN" (Commentabile)
39         Issue issueAperta = new Issue();
40         issueAperta.setIdIssue(issueId);
41         issueAperta.setStato("OPEN");
42
43         when(issueRepository.findById(issueId)).thenReturn(Optional.of(issueAperta));
44         when(commentoRepository.save(any())).thenReturn(i -> i.getArguments()[0]);
45

```





```
46     RichiestaInserimentoCommentoIssue richiesta = new
47         RichiestaInserimentoCommentoIssue();
48     richiesta.setIdIssue(issueId);
49     richiesta.setTesto("Ecco un nuovo commento!");
50
51     // ACT
52     RispostaInserimentoCommentoIssue response = service.inserisciCommento(
53         richiesta, userId);
54
55     // ASSERT
56     assertTrue(response.isSuccess(), "Il commento deve essere inserito con
57         successo");
58     verify(commentoRepository, times(1)).save(any());
59 }
60
61 @Test
62 void testInserisciCommento_SuIssueChiusa_Fallimento() {
63
64     Integer issueId = 10;
65
66     Issue issueChiusa = new Issue();
67     issueChiusa.setIdIssue(issueId);
68     issueChiusa.setStato("CLOSED");
69
70     when(issueRepository.findById(issueId)).thenReturn(Optional.of(issueChiusa));
71
72     RichiestaInserimentoCommentoIssue richiesta = new
73         RichiestaInserimentoCommentoIssue();
74     richiesta.setIdIssue(issueId);
75     richiesta.setTesto("Provo a commentare una issue chiusa...");
76
77     // ACT
78     RispostaInserimentoCommentoIssue response = service.inserisciCommento(
79         richiesta, 50L);
80
81     // ASSERT
82     assertFalse(response.isSuccess(), "L'operazione deve fallire su issue chiusa");
83     ;
84     assertEquals("Non è possibile commentare una Issue chiusa", response.
85         getMessage());
86     verify(commentoRepository, never()).save(any());
87 }
88
89 @Test
90 void testInserisciCommento_IssueNonTrovata_Fallimento() {
91     // ARRANGE
92     Integer issueId = 999;
93     Long userId = 50L;
94
95     when(issueRepository.findById(issueId)).thenReturn(Optional.empty());
96
97     RichiestaInserimentoCommentoIssue richiesta = new
98         RichiestaInserimentoCommentoIssue();
99     richiesta.setIdIssue(issueId);
100     richiesta.setTesto("Commento");
101
102     // ACT
103     RispostaInserimentoCommentoIssue response = service.inserisciCommento(
104         richiesta, userId);
105
106     // ASSERT
107     assertFalse(response.isSuccess(), "Deve fallire se l'issue non esiste");
108     assertEquals("Issue non trovata", response.getMessage());
109     verify(commentoRepository, never()).save(any());
110 }
111 }
```



```
102
103
104 @Test
105 void testInserisciCommento_IdIssueNull_Fallimento() {
106     // ARRANGE
107     Long userId = 50L;
108     RichiestaInserimentoCommentoIssue richiesta = new
109         RichiestaInserimentoCommentoIssue();
110     richiesta.setIdIssue(null);
111     richiesta.setTesto("Commento valido");
112
113     // ACT
114     RispostaInserimentoCommentoIssue response = service.inserisciCommento(
115         richiesta, userId);
116
117     // ASSERT
118     assertFalse(response.isSuccess(), "Deve fallire se l'ID issue è null");
119     assertEquals("ID Issue è obbligatorio", response.getMessage());
120     verify(commentoRepository, never()).save(any());
121 }
122
123 @Test
124 void testInserisciCommento_UtenteNonAutenticato_Fallimento() {
125     // ARRANGE
126     Integer issueId = 10;
127     RichiestaInserimentoCommentoIssue richiesta = new
128         RichiestaInserimentoCommentoIssue();
129     richiesta.setIdIssue(issueId);
130     richiesta.setTesto("Commento valido");
131
132     RispostaInserimentoCommentoIssue response = service.inserisciCommento(
133         richiesta, null);
134
135     // ASSERT
136     assertFalse(response.isSuccess(), "Deve fallire se l'utente non è autenticato");
137     assertEquals("Utente non autenticato", response.getMessage());
138     verify(commentoRepository, never()).save(any());
139 }
140
141 @Test
142 void testInserisciCommento_TestoVuoto_Fallimento() {
143     // ARRANGE
144     Integer issueId = 10;
145     Long userId = 50L;
146
147     RichiestaInserimentoCommentoIssue richiesta = new
148         RichiestaInserimentoCommentoIssue();
149     richiesta.setIdIssue(issueId);
150     richiesta.setTesto("");
151
152     // ACT
153     RispostaInserimentoCommentoIssue response = service.inserisciCommento(
154         richiesta, userId);
155
156     // ASSERT
157     assertFalse(response.isSuccess(), "Deve fallire se il testo è vuoto");
158     assertEquals("Il testo del commento non può essere vuoto", response.getMessage());
159     verify(commentoRepository, never()).save(any());
160 }
161 }
```

Listing 5.2: Test Inserimento Commento



## 5.4 Testing dell'Interfaccia Grafica (GUI Testing)

Oltre alla logica di business, abbiamo ritenuto critico testare in modo automatico anche il comportamento del Client Desktop sviluppato in JavaFX. Per farlo, abbiamo utilizzato la libreria **TestFX**, che permette di simulare interazioni utente (click, scrittura) e asserire lo stato dei nodi grafici (bottoni, label) all'interno di un ambiente JUnit.

Di seguito riportiamo il test implementato per il `LoginController`. Questo test verifica la logica di validazione della form e la corretta visualizzazione degli errori, utilizzando un Mock del servizio API per isolare il test da reali chiamate di rete.

```
1 package it.unina.bugboard.login;
2
3
4 import it.unina.bugboard.app.BugBoard;
5 import it.unina.bugboard.common.SessionManager;
6 import it.unina.bugboard.navigation.SceneRouter;
7 import javafx.fxml.FXMLLoader;
8 import javafx.scene.Parent;
9 import javafx.scene.Scene;
10 import javafx.stage.Stage;
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.mockito.Mock;
14 import org.mockito.Mockito;
15 import org.mockito.junit.jupiter.MockitoExtension;
16 import org.testfx.api.FxAssert;
17 import org.testfx.framework.junit5.ApplicationTest;
18 import org.testfx.matcher.base.NodeMatchers;
19 import org.testfx.matcher.control.LabeledMatchers;
20 import org.testfx.util.WaitForAsyncUtils;
21
22 import java.io.IOException;
23 import java.util.concurrent.TimeUnit;
24
25 import static org.mockito.Mockito.when;
26
27 @ExtendWith(MockitoExtension.class)
28 public class LoginControllerTest extends ApplicationTest {
29
30     @Mock
31     private LoginApiService loginApiService;
32
33     @Mock
34     private SessionManager sessionManager;
35
36     private LoginController controller;
37
38     @Override
39     public void start(Stage stage) throws IOException {
40         SceneRouter.inizializza(stage);
41
42         FXMLLoader loader = new FXMLLoader(BugBoard.class.getResource("/it/unina/
43             bugboard/fxml/login.fxml"));
44
45         controller = new LoginController(loginApiService, sessionManager);
46
47         loader.setControllerFactory(param -> {
48             if (param == LoginController.class) {
49                 return controller;
50             }
51             try {
52                 return param.getDeclaredConstructor().newInstance();
53             } catch (Exception e) {
54                 throw new RuntimeException(e);
55             }
56         });
57     }
58 }
```



```

54         }
55     });
56
57     Parent root = loader.load();
58     Scene scene = new Scene(root, 1000, 800);
59     stage.setScene(scene);
60     stage.show();
61 }
62
63 @Test
64 void testLoginButtonDisabledInitially() {
65     WaitForAsyncUtils.waitForFxEvents();
66     FxAssert.verifyThat("#pulsanteLogin", NodeMatchers.isDisabled());
67 }
68
69 @Test
70 void testLoginButtonEnabledWhenFieldsFilled() {
71     clickOn("#campoUsername").write("testuser");
72     clickOn("#campoPassword").write("password");
73
74     FxAssert.verifyThat("#pulsanteLogin", NodeMatchers.isEnabled());
75 }
76
77 @Test
78 void testLoginFailureShowsError() {
79     RispostaLogin rispostaFallita = Mockito.mock(RispostaLogin.class);
80     when(rispostaFallita.isSuccess()).thenReturn(false);
81     when(rispostaFallita.getMessage()).thenReturn("Credenziali errate");
82     when(loginApiService.login("utente_sbagliato", "password_sbagliata")).
83         thenReturn(rispostaFallita);
84     clickOn("#campoUsername").write("utente_sbagliato");
85     clickOn("#campoPassword").write("password_sbagliata");
86     WaitForAsyncUtils.sleep(1, TimeUnit.SECONDS);
87     clickOn("#pulsanteLogin");
88     WaitForAsyncUtils.waitForFxEvents();
89     FxAssert.verifyThat("#errorePassword", LabeledMatchers.hasText("Credenziali
90         errate"));
91     FxAssert.verifyThat("#errorePassword", NodeMatchers.isVisible());
92 }
93 }

```

Listing 5.3: Test Automatico GUI: Login Controller

Questo approccio ci ha permesso di validare il feedback visivo dell'applicazione (es. il messaggio "Credenziali errate") senza dover avviare l'intero stack di microservizi, garantendo test rapidi e ripetibili.

## 5.5 Test di Usabilità (Validazione Black Box)

A complemento dei test automatici, abbiamo eseguito una sessione di **Usability Testing** con un gruppo di utenti esterni al team di sviluppo.

### 5.5.1 Protocollo e Scenario del Test

Per rendere la simulazione il più realistica possibile, è stato definito uno scenario d'uso specifico. La sessione iniziava chiedendo all'utente di interagire con un software esterno (una versione del noto gioco "Snake"). Durante l'esecuzione, il gioco era programmato per bloccarsi in modo anormale.

A fronte di questo evento, all'utente è stato assegnato il compito di segnalare il malfunzionamento sulla piattaforma BugBoard, eseguendo i seguenti passaggi in autonomia:

1. **Registrazione e Login:** Creazione di un nuovo account utente per accedere al sistema.



2. **Segnalazione del Bug:** Inserimento di una nuova Issue relativa al crash di "Snake", includendo una descrizione dell'accaduto e **allegando uno screenshot** dell'errore.
3. **Interazione:** Inserimento di un commento successivo alla segnalazione creata.

### 5.5.2 Partecipanti e Obiettivi

Il campione di utenti selezionato per il test è stato composto prevalentemente da studenti in ambito universitario, con un background tecnico eterogeneo e non necessariamente esperto nell'uso di sistemi di *Issue Tracking*.

Questa scelta di campionamento è stata strategica per valutare due metriche fondamentali:

- **Intuitività (Learnability):** Verificare se il prodotto fosse facilmente comprensibile anche per utenti alla prima interazione ("First-time users").
- **Efficacia:** Valutare se gli utenti riuscissero a portare a termine il task (segnalazione completa con foto) senza richiedere supporto esterno.

L'assunto di base è che, se un utente non specializzato riesce a completare il flusso di segnalazione in tempi ragionevoli, il sistema risulterà estremamente fluido ed efficiente per l'utente target reale (sviluppatori o tester esperti).

### 5.5.3 Risultati e Osservazioni

I risultati ottenuti dai test sono stati nel complesso estremamente positivi e confermano la validità delle scelte di design. L'analisi dettagliata delle risposte e i grafici relativi alla soddisfazione degli utenti sono consultabili al seguente [link ai risultati del questionario](#).

Dall'analisi dei dati e dall'osservazione diretta sono emersi i seguenti punti chiave:

- **Tasso di Successo (Effectiveness):** Il 100% degli utenti ha completato i task critici (registrazione, segnalazione con foto, commento) senza errori bloccanti, dimostrando che il flusso principale è solido.
- **Efficienza (Time on Task):** Il tempo medio per inserire la segnalazione del bug (compreso l'upload dello screenshot) è risultato inferiore alle aspettative (< 2 minuti), indicando che l'interfaccia è snella e priva di distrazioni.
- **Gestione degli Allegati:** Il meccanismo di upload dello screenshot è stato giudicato intuitivo anche da chi non aveva familiarità con il sistema. Nessun utente ha avuto dubbi su come allegare la prova del crash di "Snake".
- **Chiarezza dei feedback:** Gli utenti hanno apprezzato i messaggi di conferma visiva (es. "Issue inserita con successo"), che hanno ridotto l'incertezza dopo l'invio della segnalazione.

### Aree di Miglioramento e Azioni Correttive

Nonostante il successo generale, il test ha evidenziato uno spunto critico di miglioramento (Formative Testing) relativo all'interazione con i campi data.

Durante la compilazione del form, un utente ha espresso frustrazione per l'impossibilità di digitare la data manualmente tramite tastiera, essendo vincolato all'utilizzo esclusivo del selettore grafico (Calendario/DatePicker). Questa rigidità rallentava l'inserimento per gli utenti più esperti ("Power Users").

**Risoluzione:** Accogliendo questo feedback, abbiamo rivisitato l'implementazione del componente rendendo possibile un approccio ibrido: ora il sistema supporta sia la selezione visuale tramite GUI sia l'inserimento testuale manuale. Questa modifica ha incrementato significativamente la flessibilità d'uso del sistema.



### 5.5.4 Nuova analisi SonarQube

A seguito dell'implementazione delle strategie di testing descritte nei paragrafi precedenti, è stata effettuata una nuova analisi della qualità del codice mediante la piattaforma SonarQube.

I risultati ottenuti mostrano un incremento significativo del valore di copertura del codice ed evidenziano l'efficacia degli interventi apportati. Tale miglioramento è attribuibile all'introduzione dei test automatizzati sull'interfaccia grafica e all'estensione della copertura sui servizi di backend che hanno permesso di validare porzioni di logica applicativa precedentemente escluse dal calcolo.

La Figura 5.1 riporta la dashboard aggiornata e conferma il raggiungimento di un livello di affidabilità e manutenibilità del sistema più elevato.

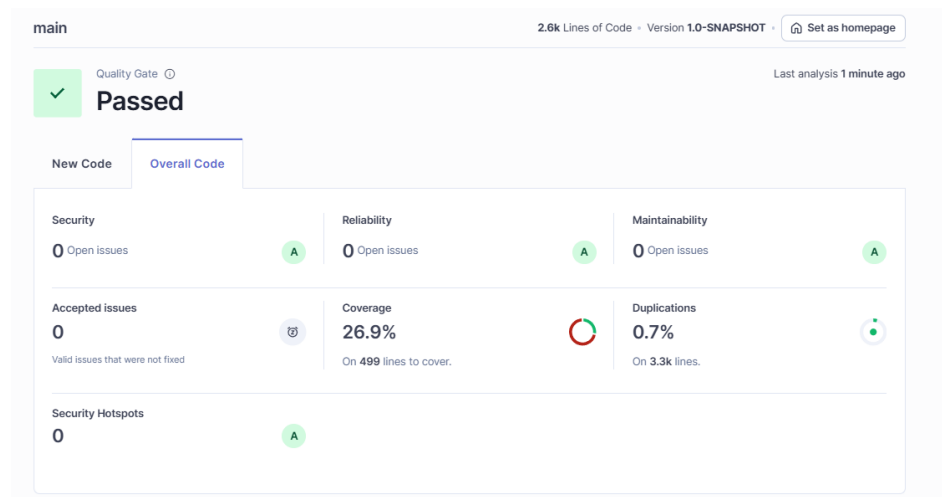


Figura 5.1: Nuova analisi SonarQube con coverage incrementata